

how to build a **wicked fast circuit optimizer**

Aws Albarghouthi | University of Wisconsin–Madison

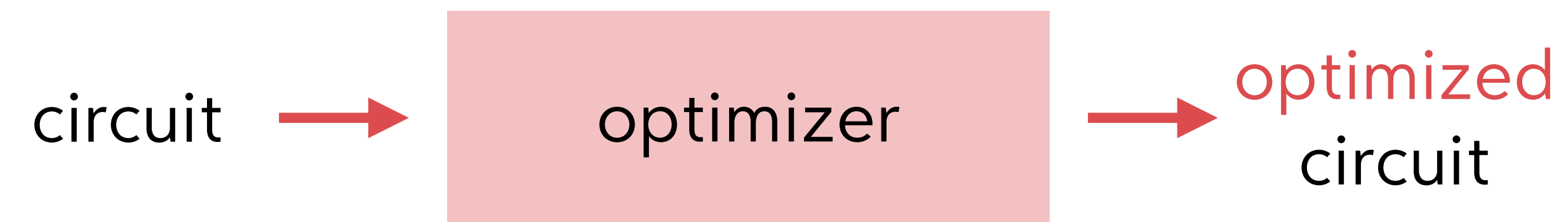


What's an optimizer anyway?

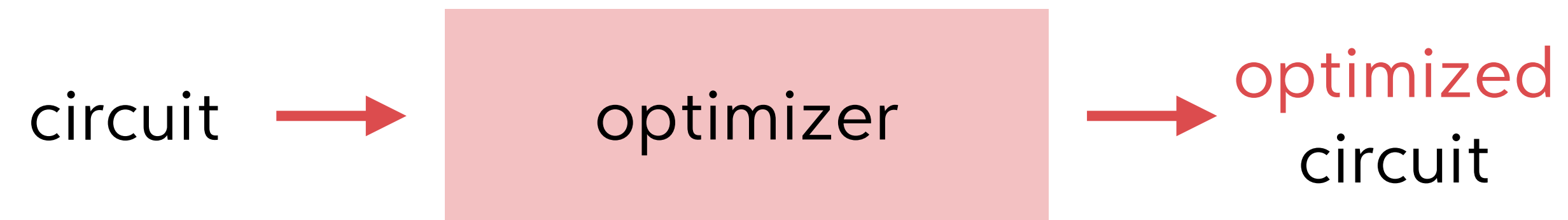
What's an optimizer anyway?



What's an optimizer anyway?

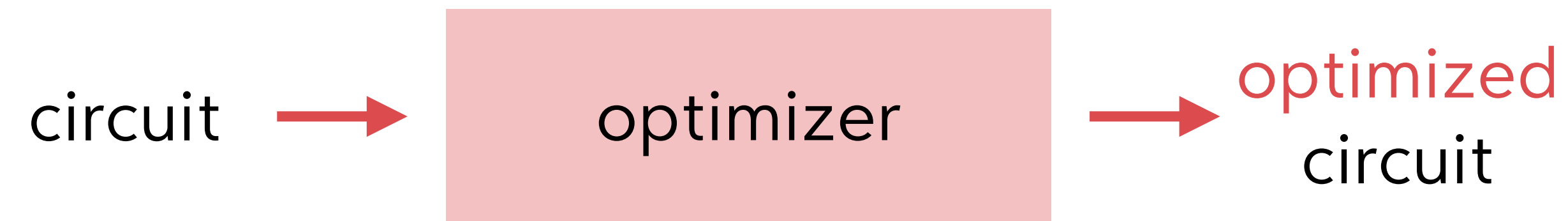


What's an optimizer anyway?



minimize resource requirements

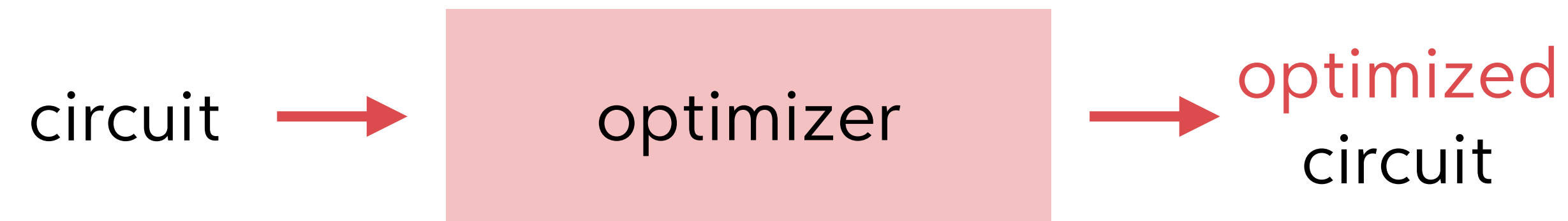
What's an optimizer anyway?



minimize resource requirements

maximize success probability

What's an optimizer anyway?



minimize resource requirements

maximize success probability

enable high-level programming

How do we build optimizers?

How do we build optimizers?

traditional optimizers

How do we build optimizers?

traditional optimizers

- most optimizers have this structure—Qiskit, tket, PennyLane, ...

How do we build optimizers?

traditional optimizers

- most optimizers have this structure—Qiskit, tket, PennyLane, ...
- sequence of classical-style compiler “**passes**”

How do we build optimizers?

traditional optimizers

- most optimizers have this structure—Qiskit, tket, PennyLane, ...
- sequence of classical-style compiler “**passes**”
- each pass is **best-effort**

How do we build optimizers?

traditional optimizers

- most optimizers have this structure—Qiskit, tket, PennyLane, ...
- sequence of classical-style compiler “**passes**”
- each pass is **best-effort**
- **pros**: fast, scalable (sometimes)

How do we build optimizers?

traditional optimizers

- most optimizers have this structure—Qiskit, tket, PennyLane, ...
- sequence of classical-style compiler “passes”
- each pass is best-effort
- pros: fast, scalable (sometimes)
- cons: far from optimal

How do we build optimizers?

superoptimizers

How do we build optimizers?

superoptimizers

- mostly academic tools—QUESO, Quasar, QuaRL, GUOQ, ...

How do we build optimizers?

superoptimizers

- mostly academic tools—QUESO, Quasar, QuaRL, GUOQ, ...
- view optimization as a **search problem**

How do we build optimizers?

superoptimizers

- mostly academic tools—QUESO, Quasar, QuaRL, GUOQ, ...
- view optimization as a **search problem**
- **combinatorial** search for best circuit

How do we build optimizers?

superoptimizers

- mostly academic tools—QUESO, Quasar, QuaRL, GUOQ, ...
- view optimization as a **search problem**
- **combinatorial** search for best circuit
- **pros**: significant reductions over traditional optimizers

How do we build optimizers?

superoptimizers

- mostly academic tools—QUESO, Quasar, QuaRL, GUOQ, ...
- view optimization as a **search problem**
- **combinatorial** search for best circuit
- **pros**: significant reductions over traditional optimizers
- **cons**: incredibly slow, and not scalable

I want to solve a trilemma. I want a:

I want to solve a **trilemma**. I want a:

1. fast

I want to solve a **trilemma**. I want a:

1. fast

2. scalable

I want to solve a **trilemma**. I want a:

1. fast

2. scalable

3. superoptimizer

~/iwqc

> **tzap** hamiltonian-simulation-n7.qasm

⚡ **tzap** v0.6.1

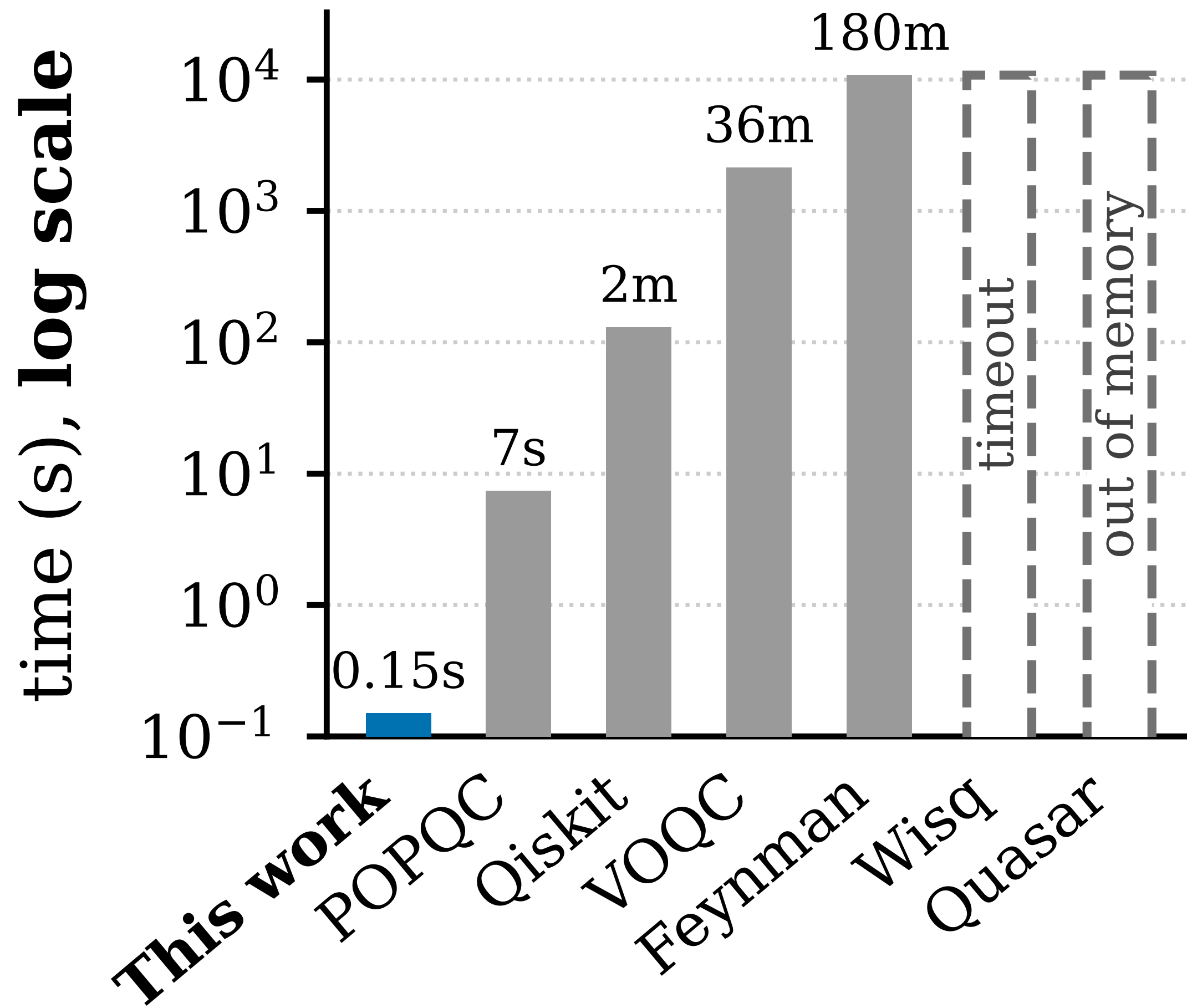
Parsed hamiltonian-simulation-n7.qasm (1.6 MB) in 0.022s

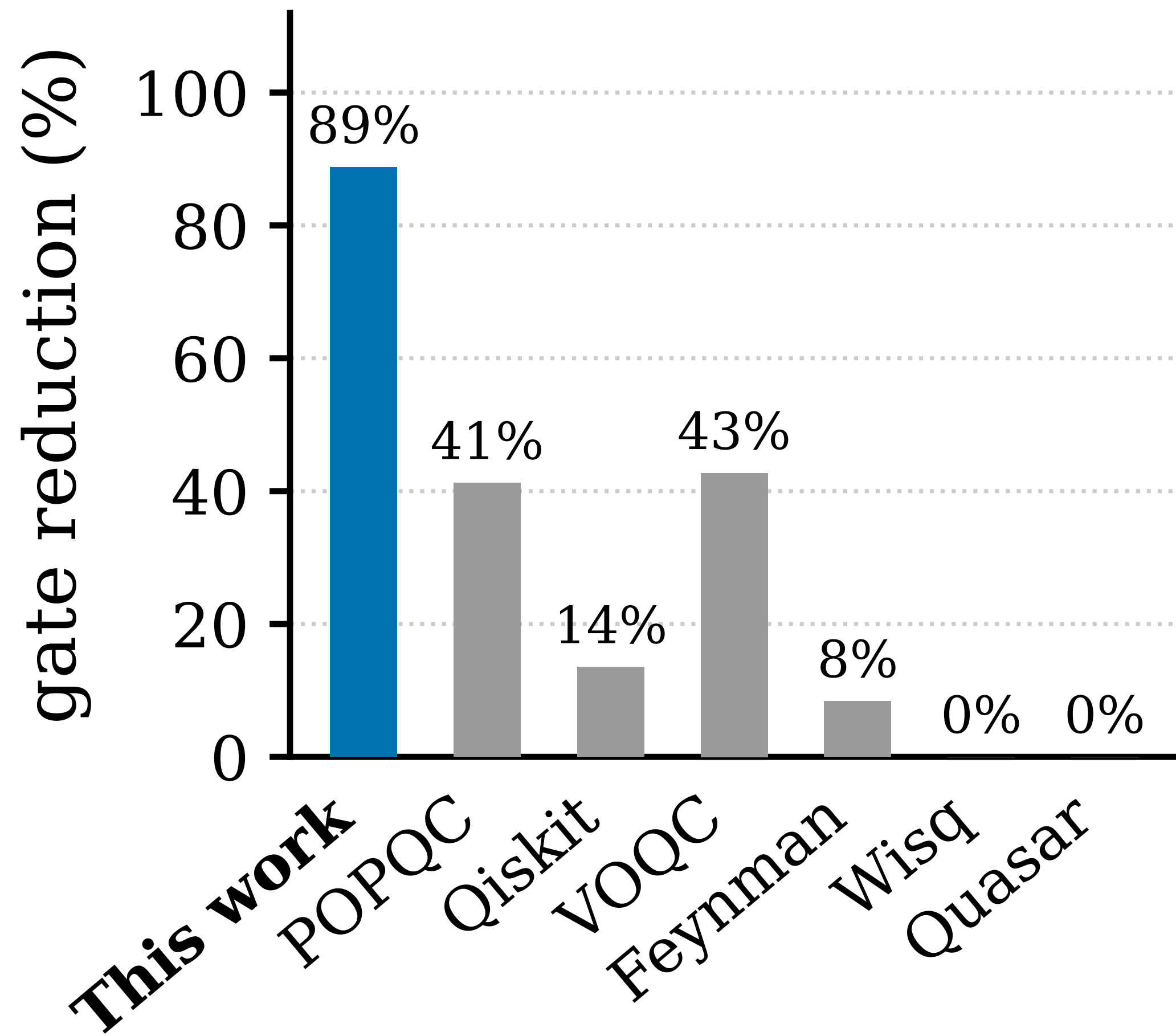
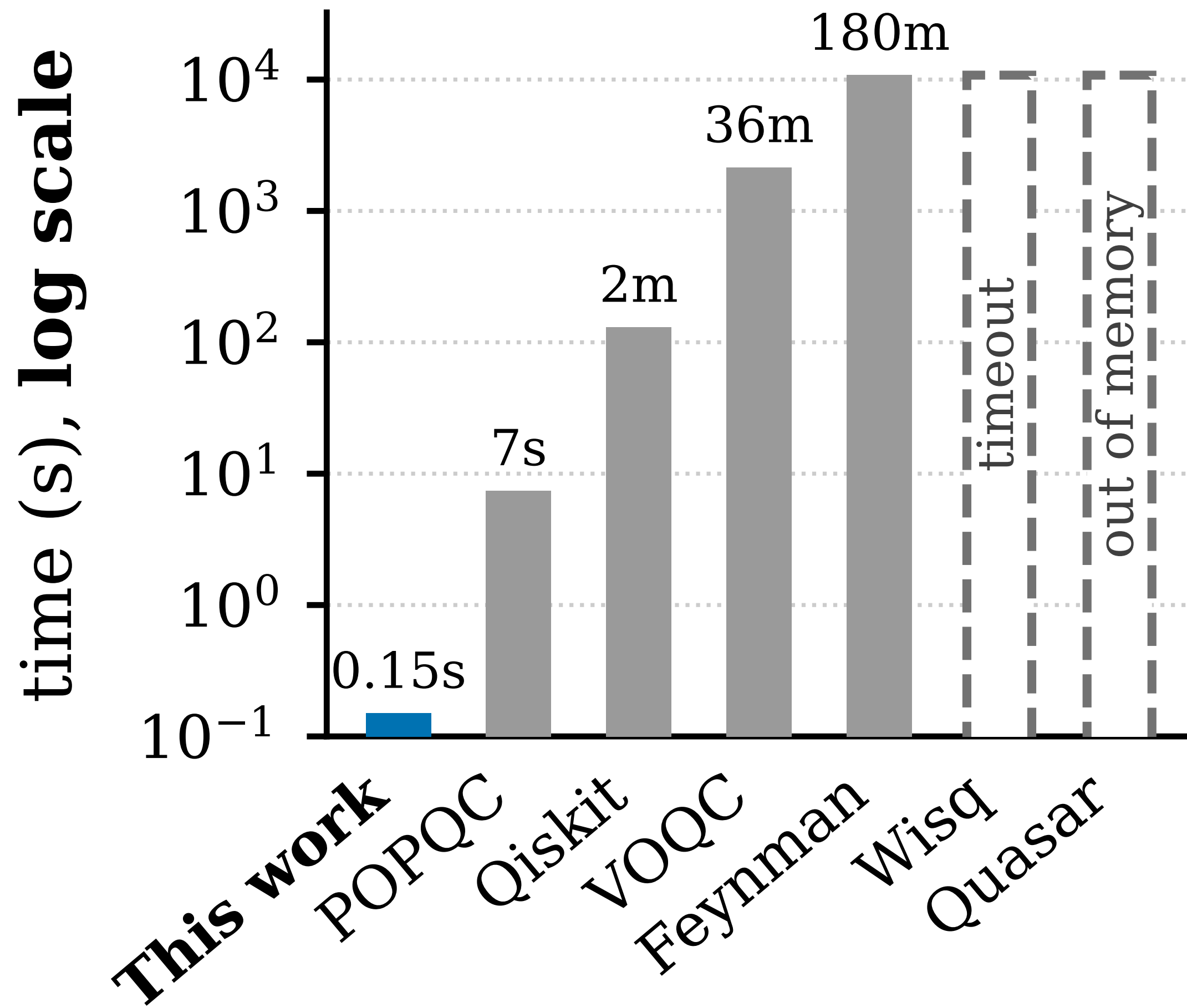
└─ 16 qubits · 142,910 gates

Loaded superoptimizer table in 0.016s

Converged after 9 rounds

Final result · 88.8% fewer gates · 0.153s				
Gates	_____	↓88.8%	· 142,910 →	16,029
2q gates	_____	↓86.7%	· 56,764 →	7,564
T/Tdg	_____	↓91.7%	· 66,143 →	5,515
Rz	_____	↓47.1%	· 323 →	171
Depth	_____	↓81.9%	· 72,530 →	13,110

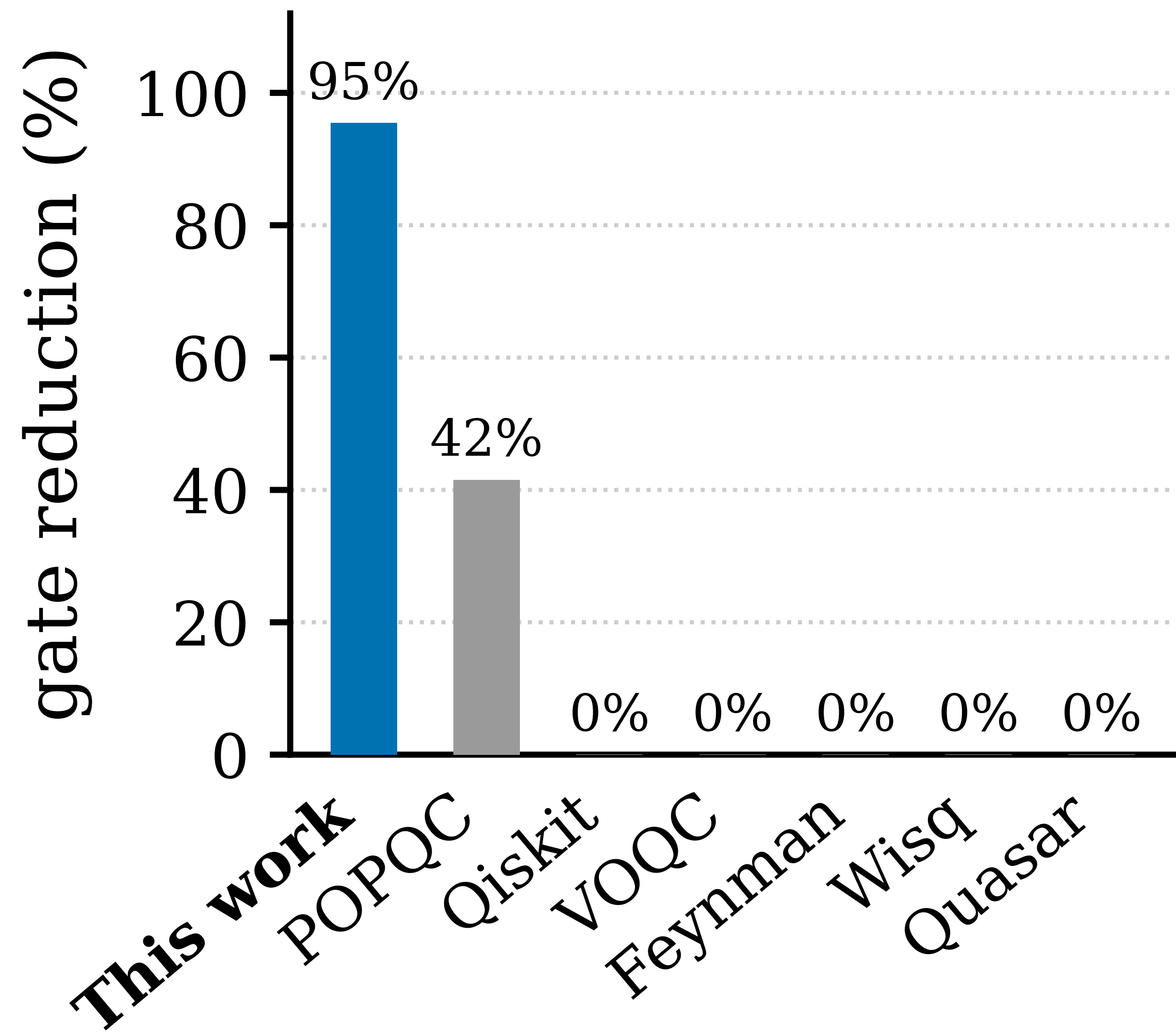
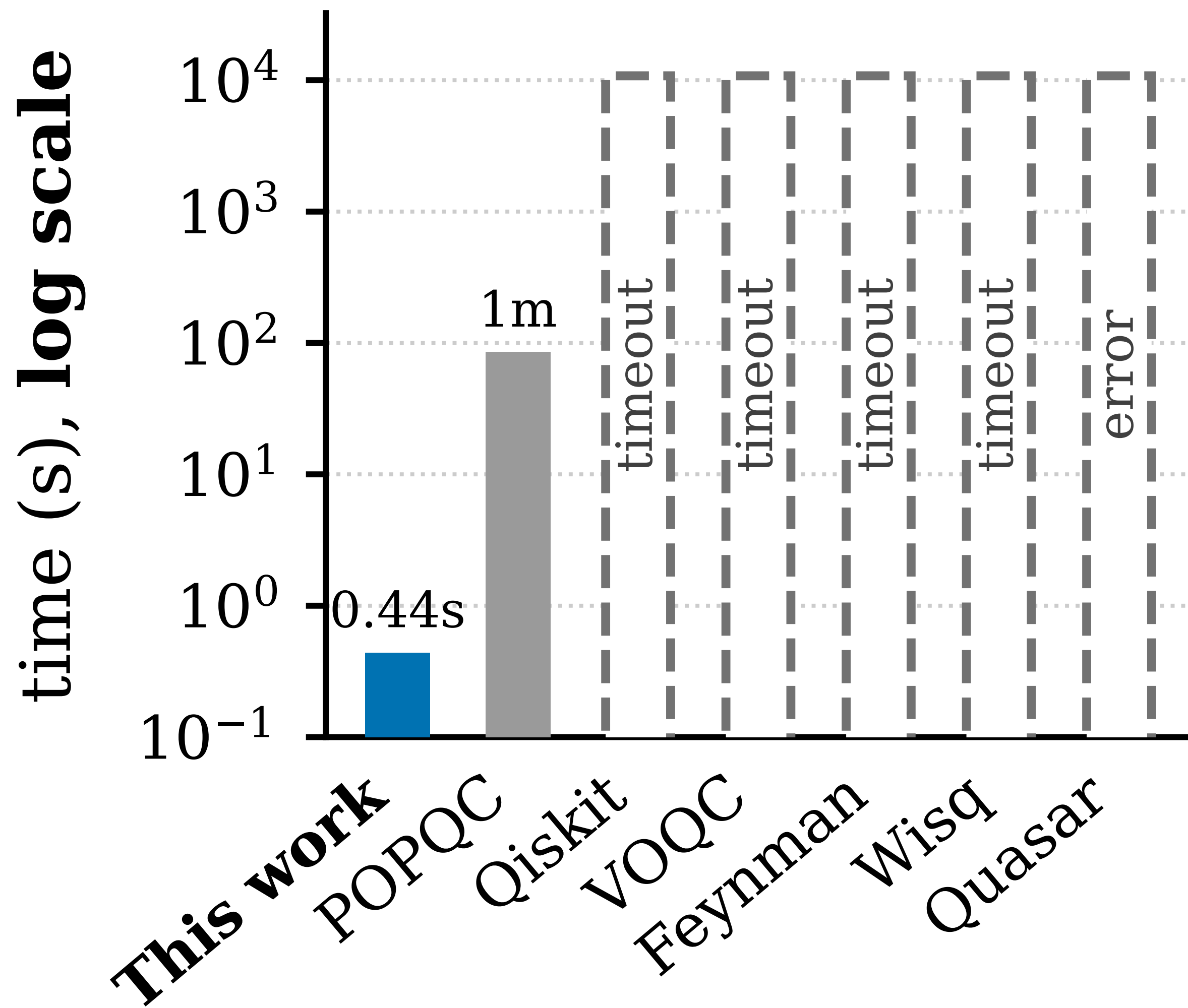




```
~/iwqc
> tzap hamiltonian-simulation-n13.qasm
⚡ tzap v0.6.1
  Parsed hamiltonian-simulation-n13.qasm (22.2 MB) in 0.165s
    └─ 20 qubits · 1,955,569 gates
  Optimizing chunks in parallel (1,000,000+ gates) · --no-parallel to disable
  Loaded superoptimizer table in 0.011s
```

```
└─ Final result · 95.4% fewer gates · 0.452s ───────────────────────────────────
```

Gates	──────────────────────────────────	↓95.4%	·	1,955,569	→	90,605
2q gates	──────────────────────────────────	↓94.4%	·	780,839	→	43,366
T/Tdg	──────────────────────────────────	↓96.5%	·	910,833	→	31,431
Rz	──────────────────────────────────	↓41.2%	·	817	→	480
Depth	──────────────────────────────────	↓92.3%	·	982,117	→	75,202



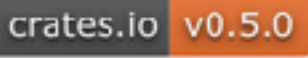
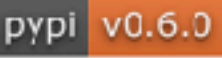
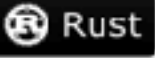
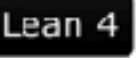
Introducing tzap

Clifford+Rz circuit optimizer

tzap's philosophy



tzap

 CI **passing**  crates.io **v0.5.0**  pypi **v0.6.0**  Rust  Lean 4  license **Apache 2.0**  arXiv **2605.13929**

[Installation](#) · [Using tzap](#) | [Qiskit integration](#) · [PennyLane integration](#)

A super fast, Rust-based optimizer for large Clifford+T circuits.

- tzap is state-of-the-art in *speed*, *scalability*, and *gate-count reduction*.
- tzap **minimizes T-count** with a new linear-time phase folding algorithm, based on [this paper](#).
- tzap implements a new and fast **superoptimization** pass.
- A **formally verified** Lean port of tzap is available in [lean](#).

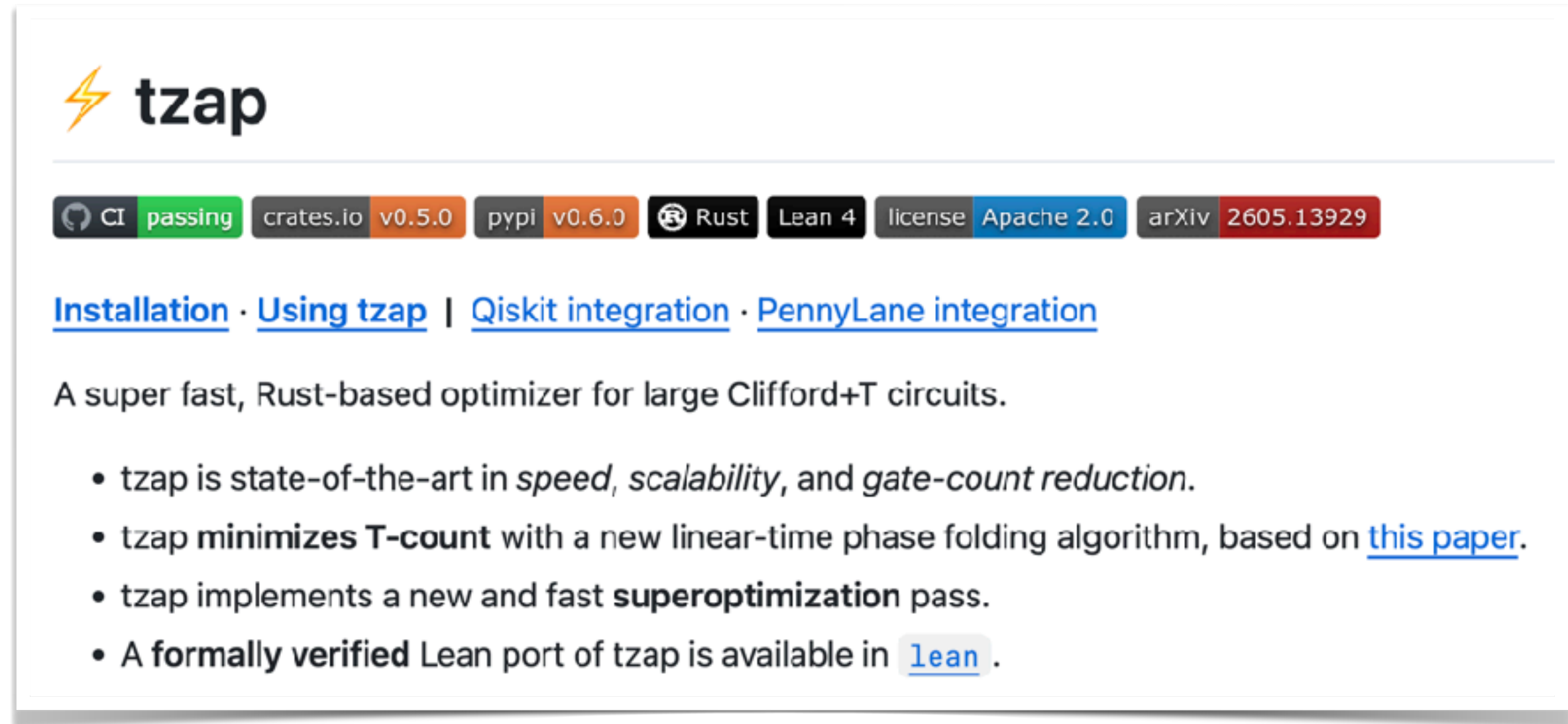
github.com/qqq-wisc/tzap

Introducing tzap

Clifford+Rz circuit optimizer

tzap's philosophy

- built like a traditional optimizer



The screenshot shows the GitHub repository page for 'tzap'. At the top is the 'tzap' logo with a lightning bolt icon. Below it is a row of status badges: 'CI passing' (green), 'crates.io v0.5.0' (orange), 'pypi v0.6.0' (orange), 'Rust' (black), 'Lean 4' (black), 'license Apache 2.0' (blue), and 'arXiv 2605.13929' (red). Below the badges are links for 'Installation', 'Using tzap', 'Qiskit integration', and 'PennyLane integration'. The main text describes tzap as a 'super fast, Rust-based optimizer for large Clifford+T circuits.' It lists four bullet points: tzap is state-of-the-art in speed, scalability, and gate-count reduction; tzap minimizes T-count with a new linear-time phase folding algorithm based on a paper; tzap implements a new and fast superoptimization pass; and a formally verified Lean port of tzap is available in 'lean'.

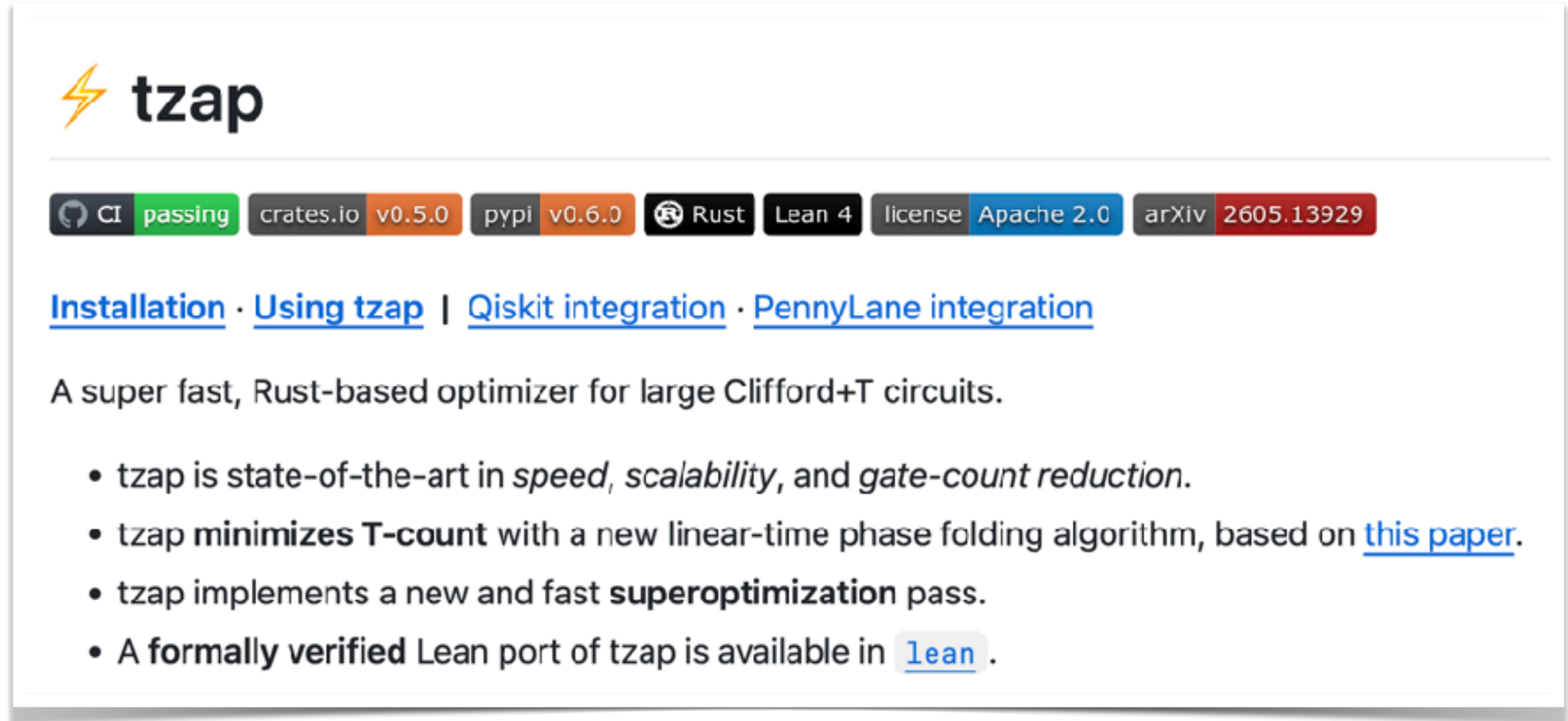
github.com/qqq-wisc/tzap

Introducing tzap

Clifford+Rz circuit optimizer

tzap's philosophy

- built like a traditional optimizer
- each pass is provably **linear** in circuit size—**very fast**



The screenshot shows the GitHub repository page for 'tzap'. At the top is the 'tzap' logo with a lightning bolt icon. Below it is a row of badges: 'CI passing' (green), 'crates.io v0.5.0' (orange), 'pypi v0.6.0' (orange), 'Rust' (black), 'Lean 4' (black), 'license Apache 2.0' (blue), and 'arXiv 2605.13929' (red). Below the badges are links for 'Installation', 'Using tzap', 'Qiskit integration', and 'PennyLane integration'. The main text describes tzap as a 'super fast, Rust-based optimizer for large Clifford+T circuits.' It lists four bullet points: tzap is state-of-the-art in speed, scalability, and gate-count reduction; tzap minimizes T-count with a new linear-time phase folding algorithm based on a paper; tzap implements a new and fast superoptimization pass; and a formally verified Lean port of tzap is available in 'lean'.

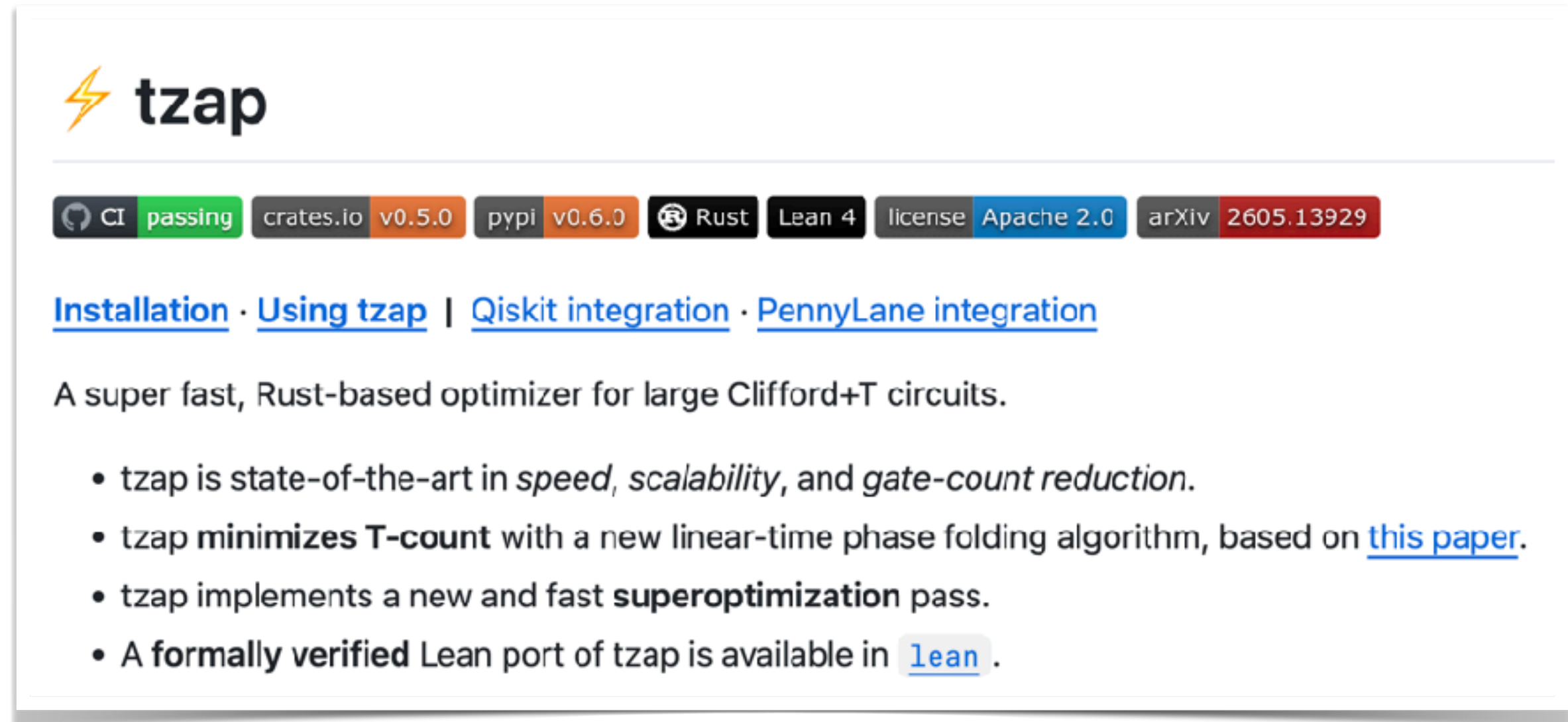
github.com/qqq-wisc/tzap

Introducing tzap

Clifford+Rz circuit optimizer

tzap's philosophy

- built like a traditional optimizer
- each pass is provably **linear** in circuit size—**very fast**
- each pass should be very **aggressive**



The screenshot shows the GitHub repository page for 'tzap'. At the top is the 'tzap' logo with a lightning bolt icon. Below it is a row of badges: 'CI passing' (green), 'crates.io v0.5.0' (orange), 'pypi v0.6.0' (orange), 'Rust' (black), 'Lean 4' (black), 'license Apache 2.0' (blue), and 'arXiv 2605.13929' (red). Below the badges are links for 'Installation', 'Using tzap', 'Qiskit integration', and 'PennyLane integration'. The main text describes tzap as a 'super fast, Rust-based optimizer for large Clifford+T circuits.' It lists four bullet points: 'tzap is state-of-the-art in speed, scalability, and gate-count reduction.', 'tzap minimizes T-count with a new linear-time phase folding algorithm, based on this paper.', 'tzap implements a new and fast superoptimization pass.', and 'A formally verified Lean port of tzap is available in lean.' The URL 'github.com/qqq-wisc/tzap' is displayed at the bottom right of the screenshot.

github.com/qqq-wisc/tzap

How to build a wicked fast optimizer

How to build a wicked fast optimizer

Chapter 1: Fast phase folding

How to build a wicked fast optimizer

Chapter 1: Fast phase folding

Chapter 2: Fast superoptimization

How to build a wicked fast optimizer

Chapter 1: Fast phase folding

Chapter 2: Fast superoptimization

Epilogue: On correctness

Chapter 1

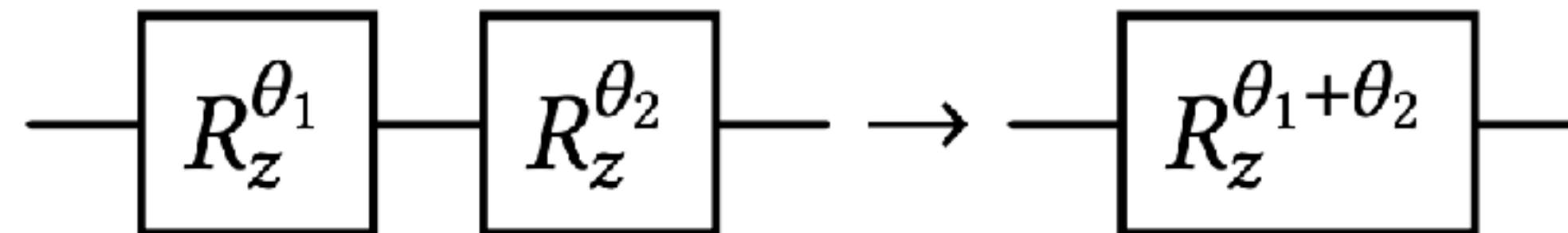
Fast phase folding

What is phase folding?

an optimization for merging rotations

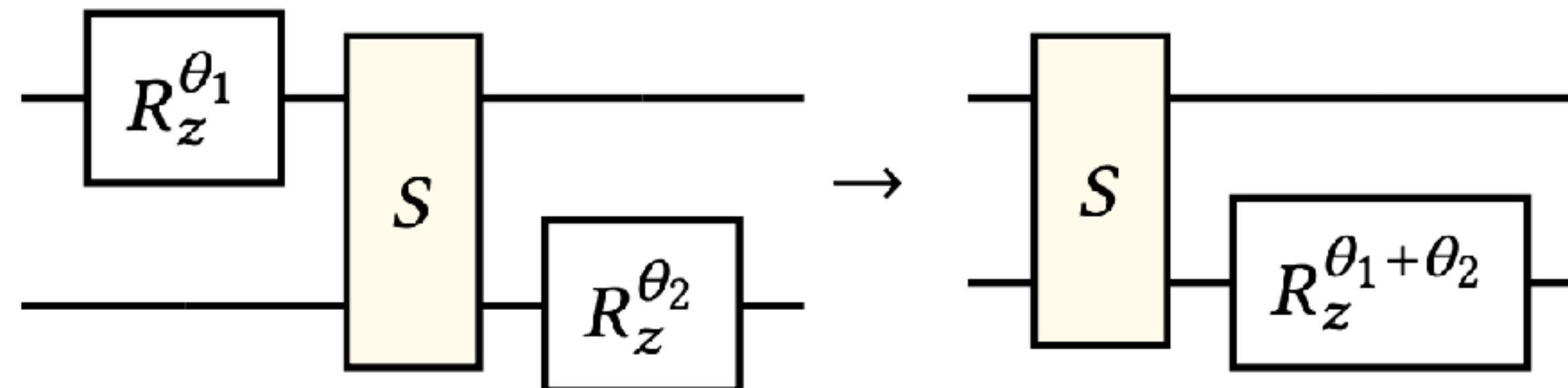
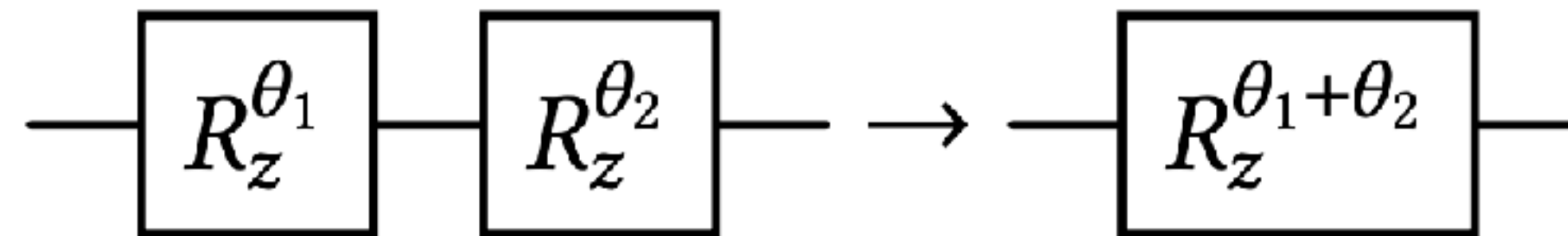
What is phase folding?

an optimization for merging rotations



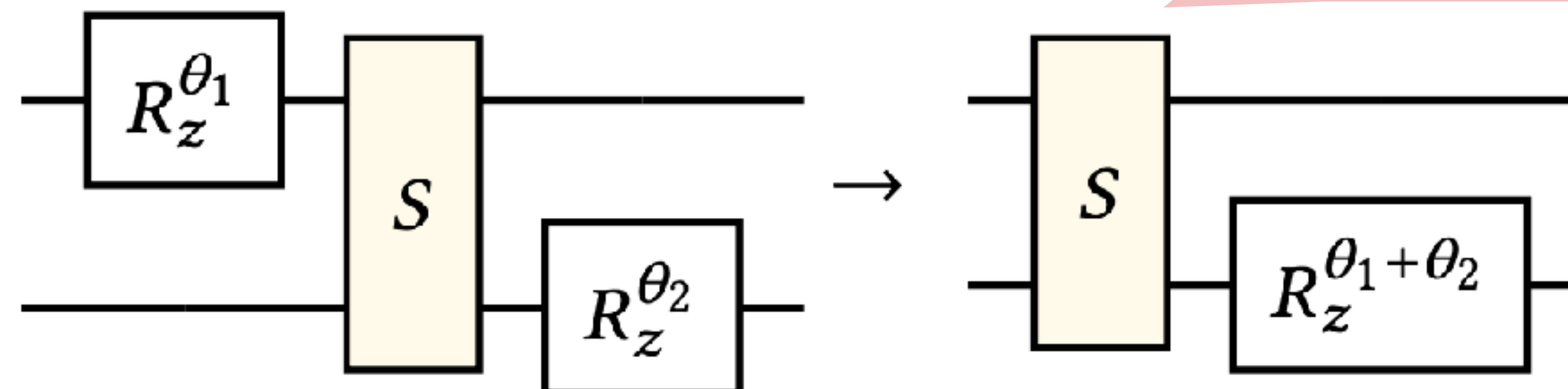
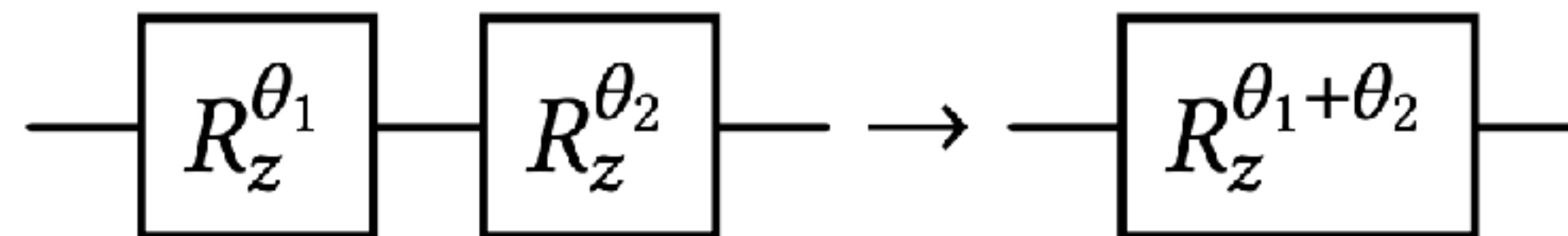
What is phase folding?

an optimization for **merging rotations**



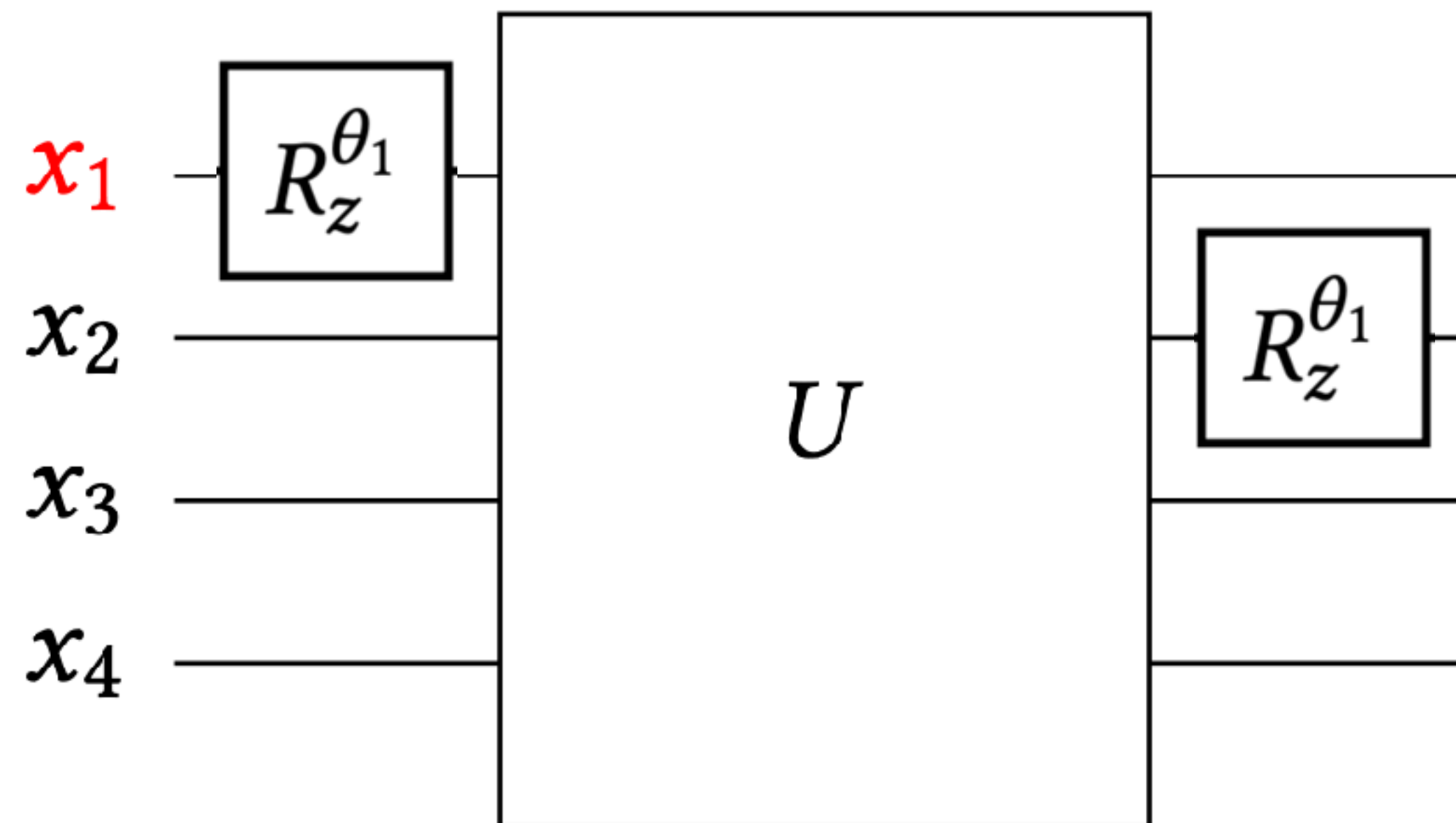
What is phase folding?

an optimization for **merging rotations**



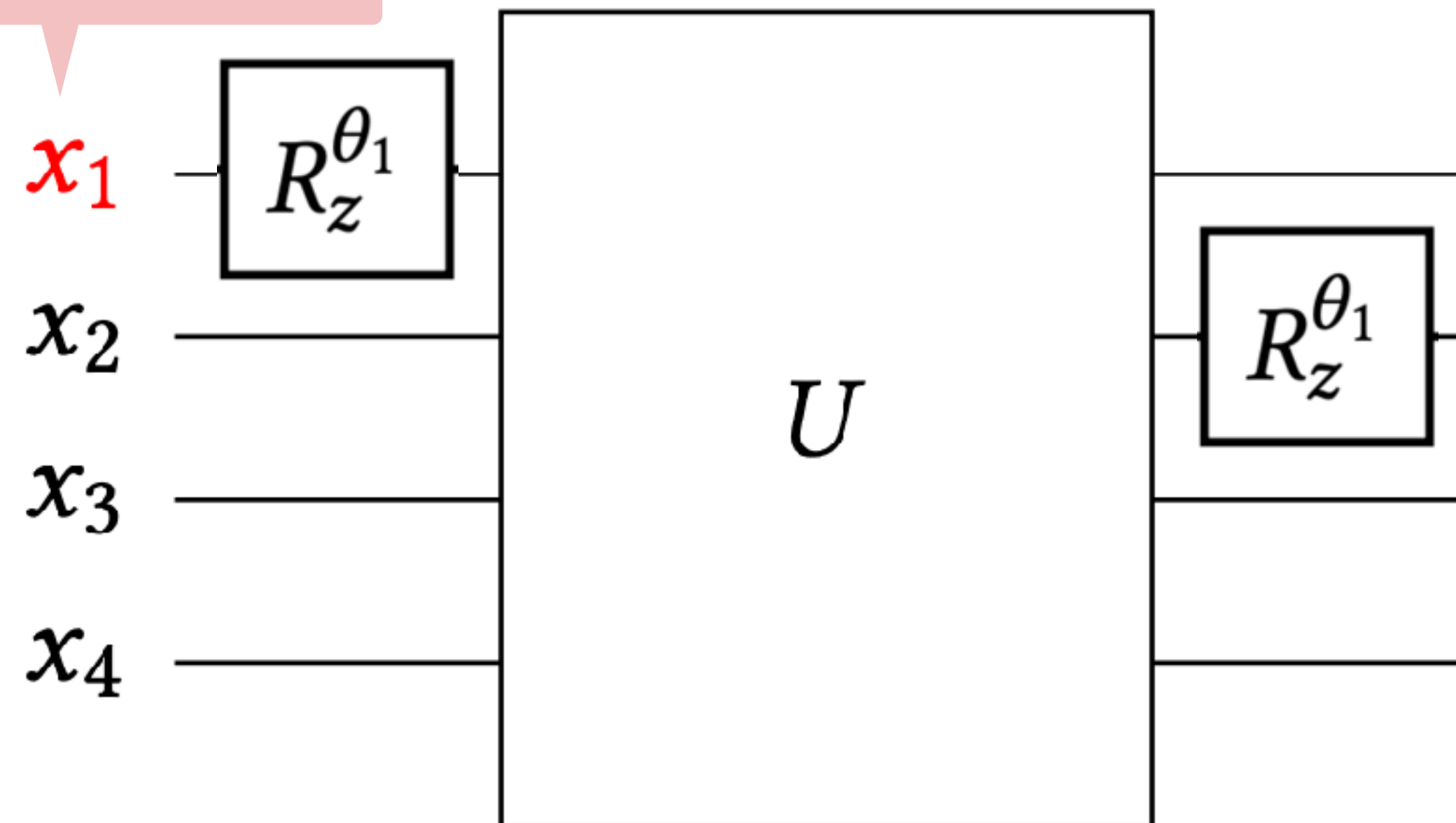
under what conditions can I merge?

When is phase folding correct?



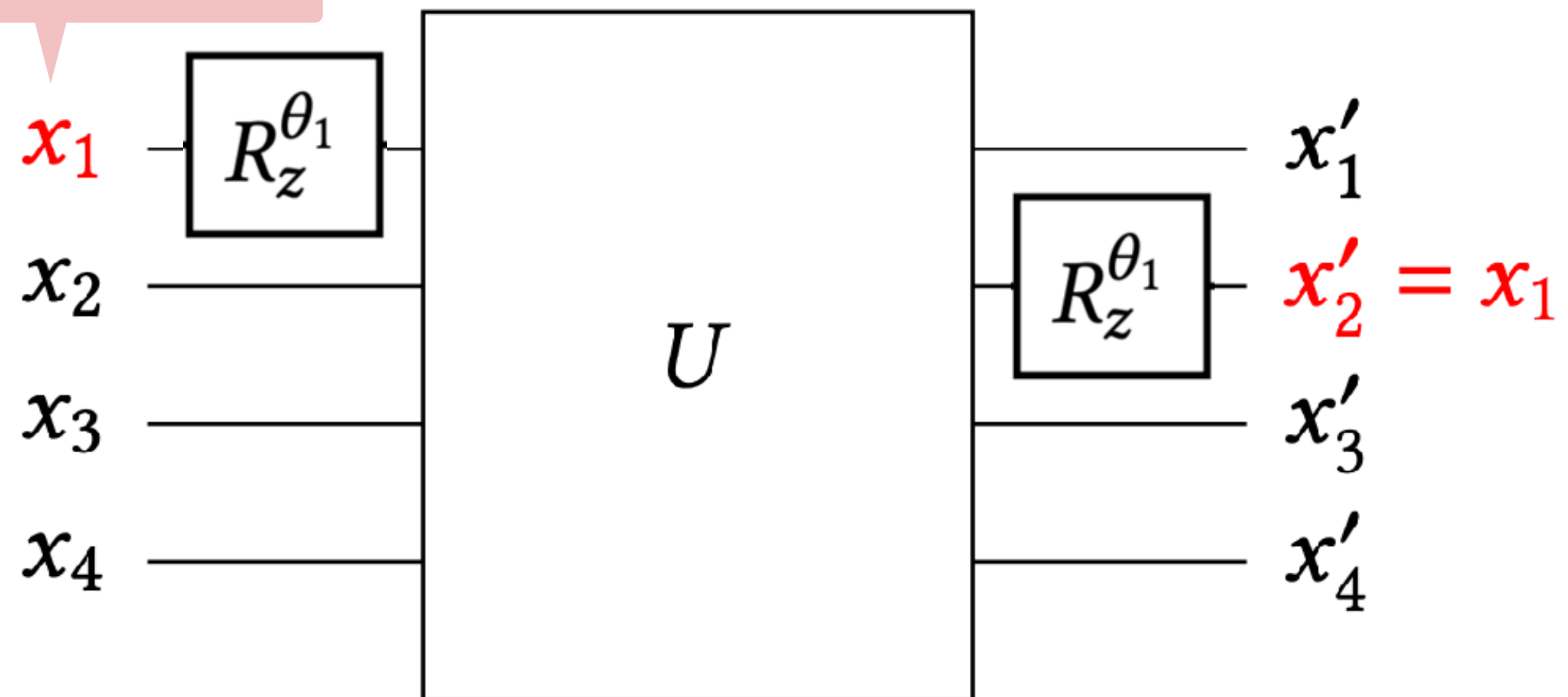
When is phase folding correct?

start with an arbitrary basis state



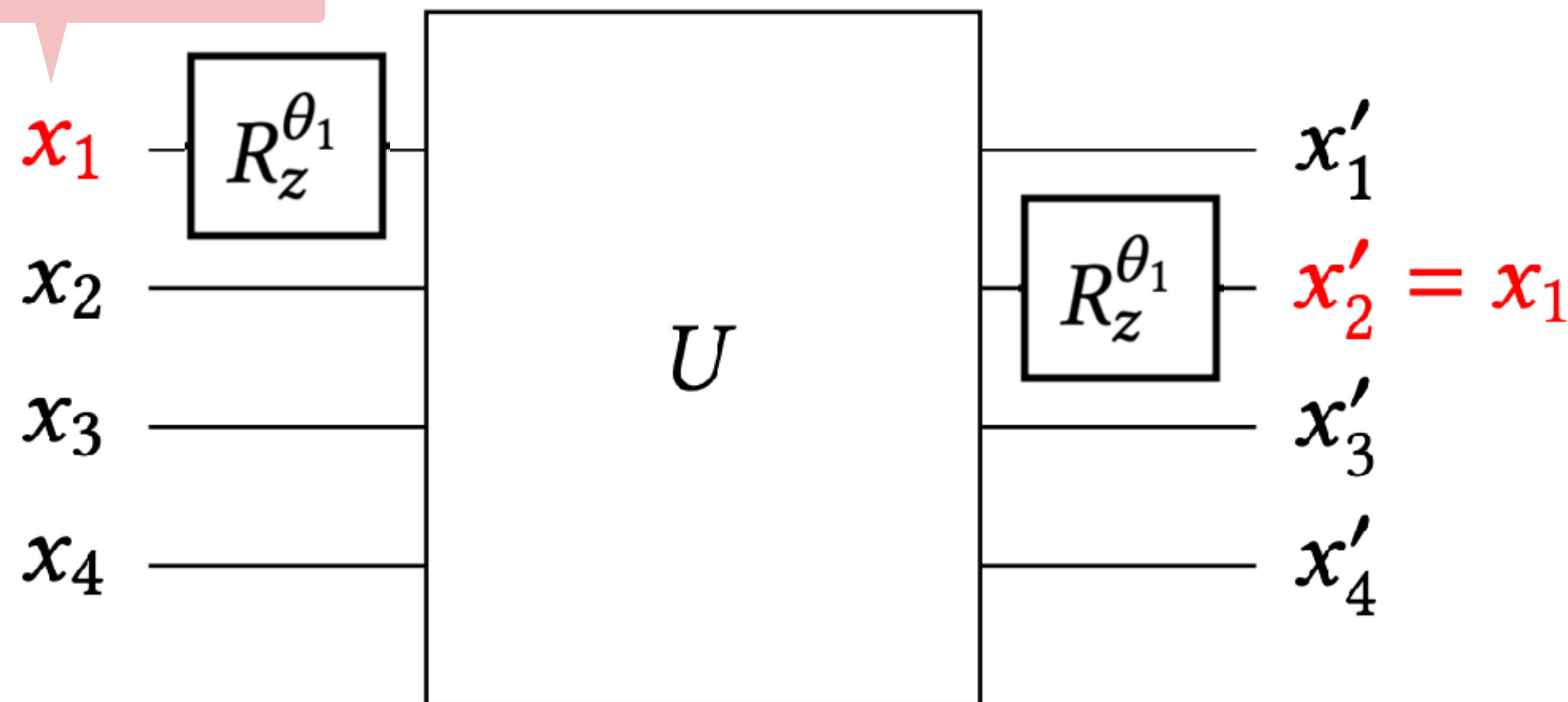
When is phase folding correct?

start with an arbitrary basis state



When is phase folding correct?

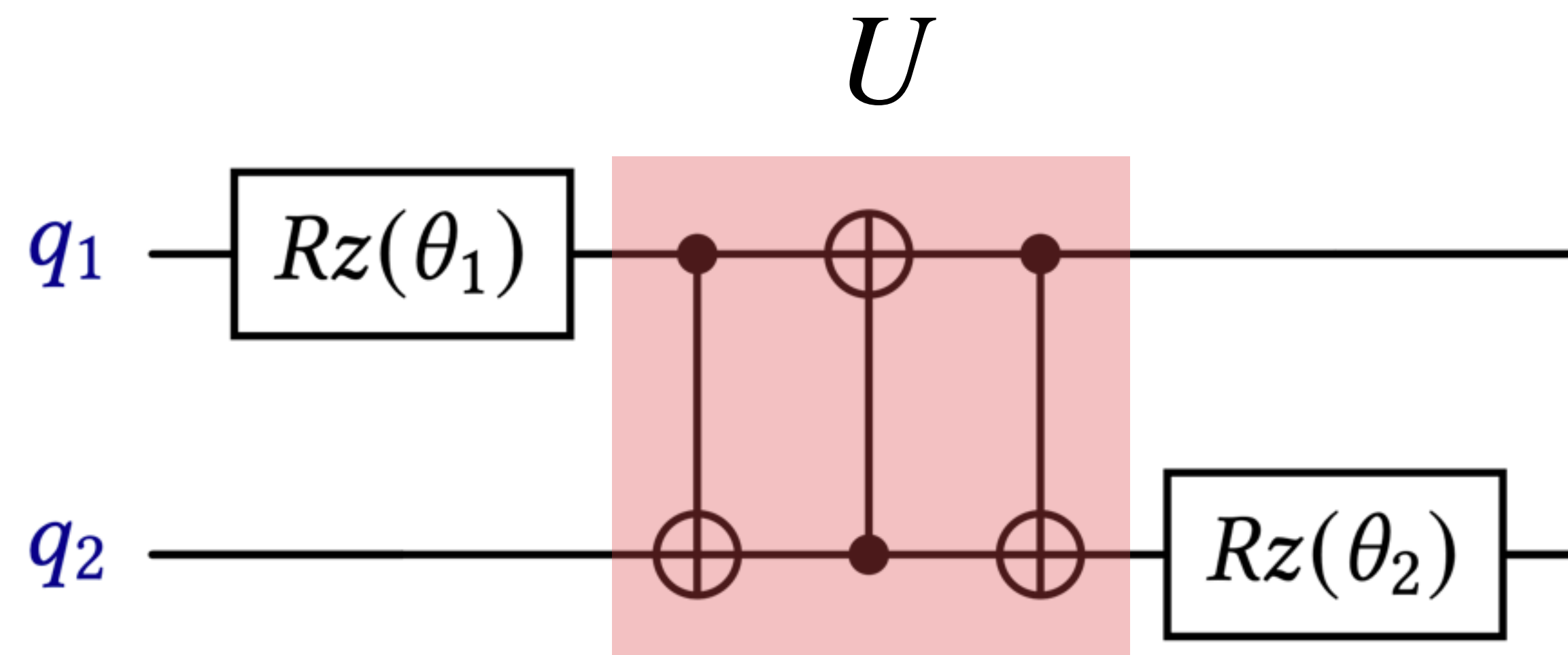
start with an arbitrary basis state



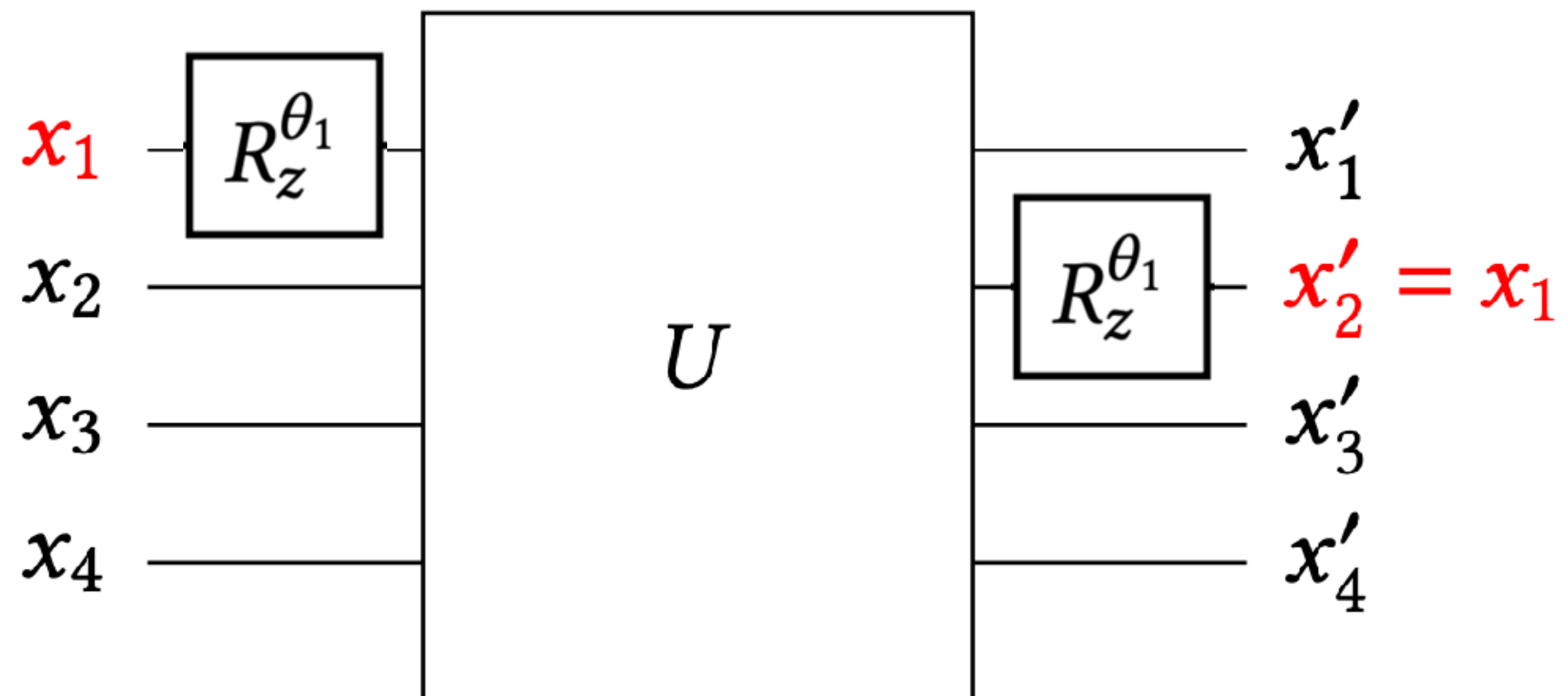
Polynomial-time T-depth Optimization of Clifford+T circuits via
Matroid Partitioning

Matthew Amy,¹ Dmitri Maslov,² and Michele Mosca^{3,4}

Ex: phase folding

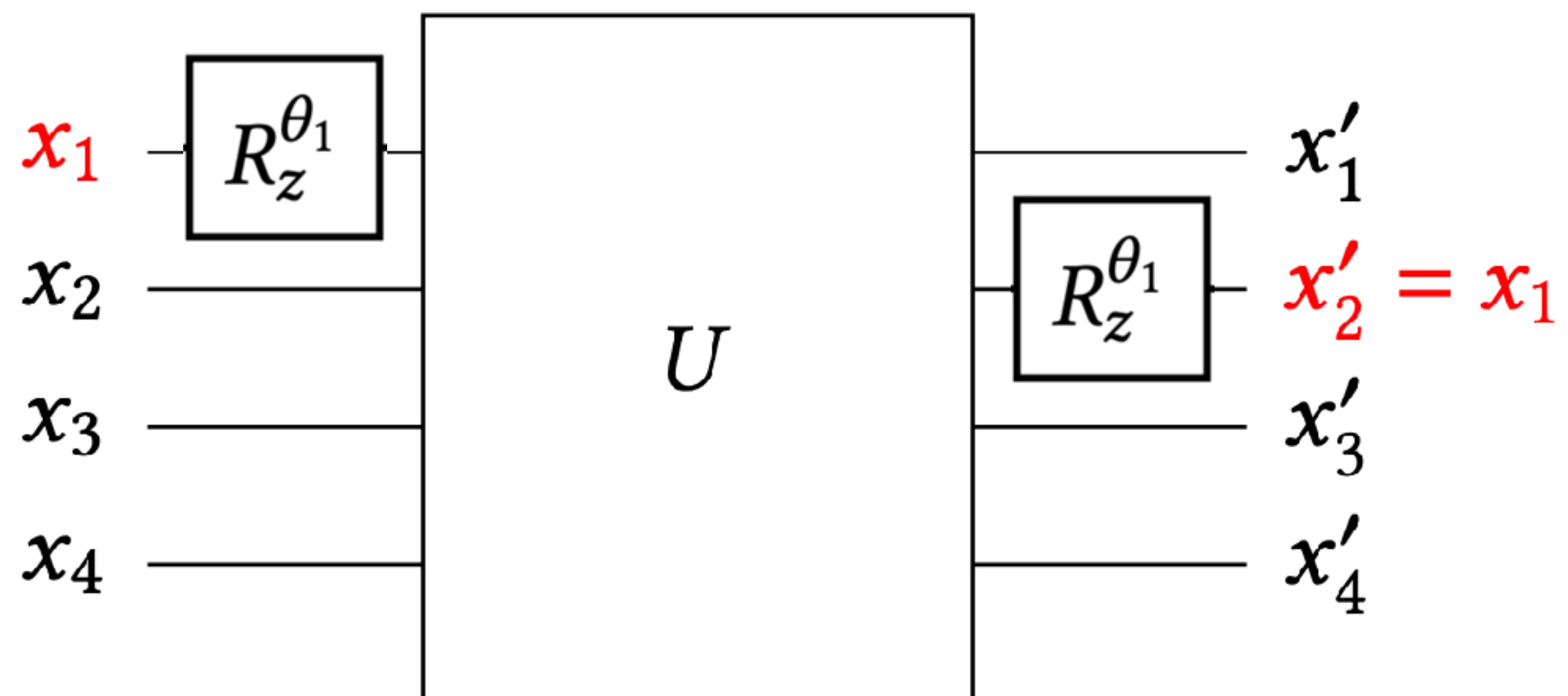


How do we check this property?

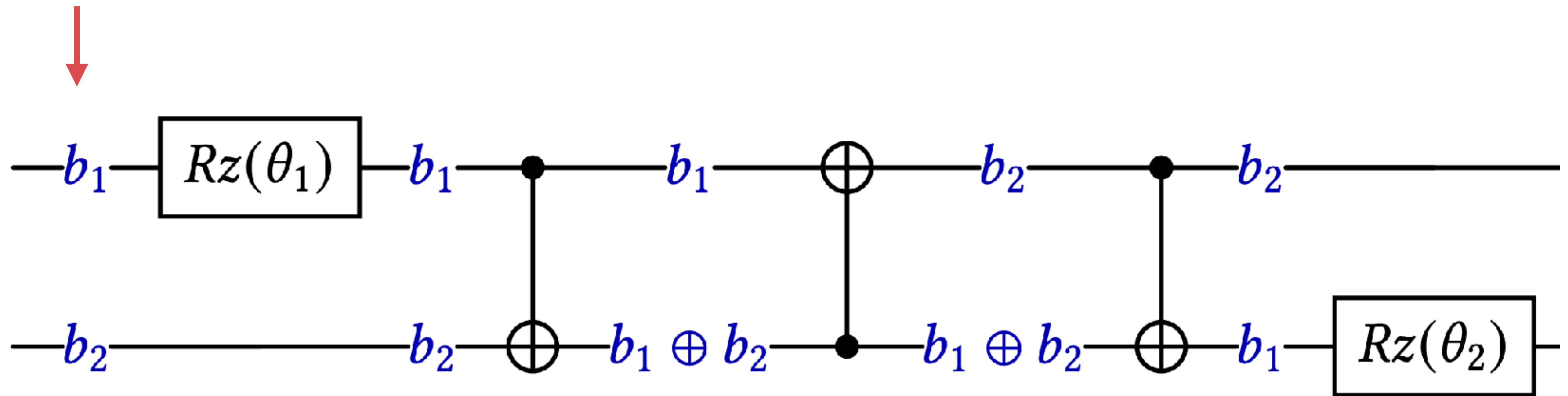


How do we check this property?

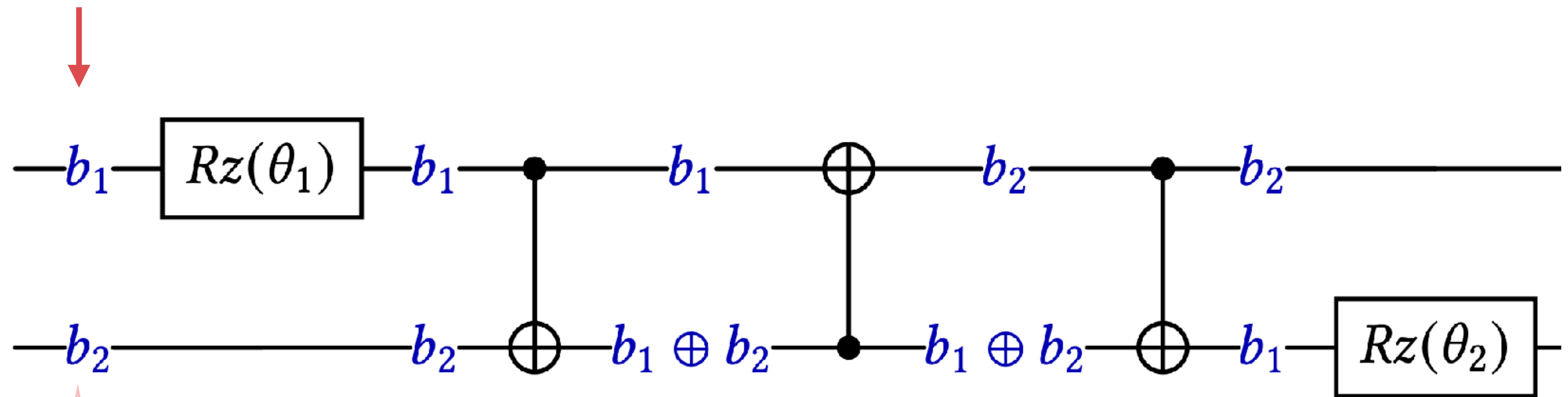
via static **program analysis**



Symbolic analysis

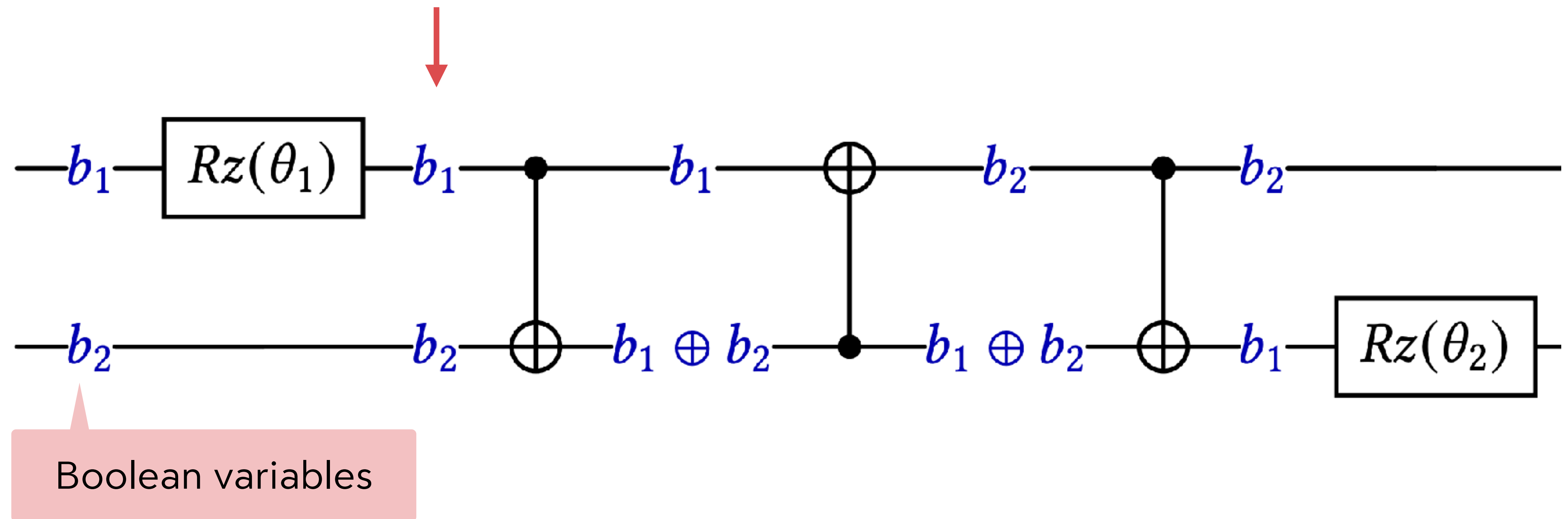


Symbolic analysis

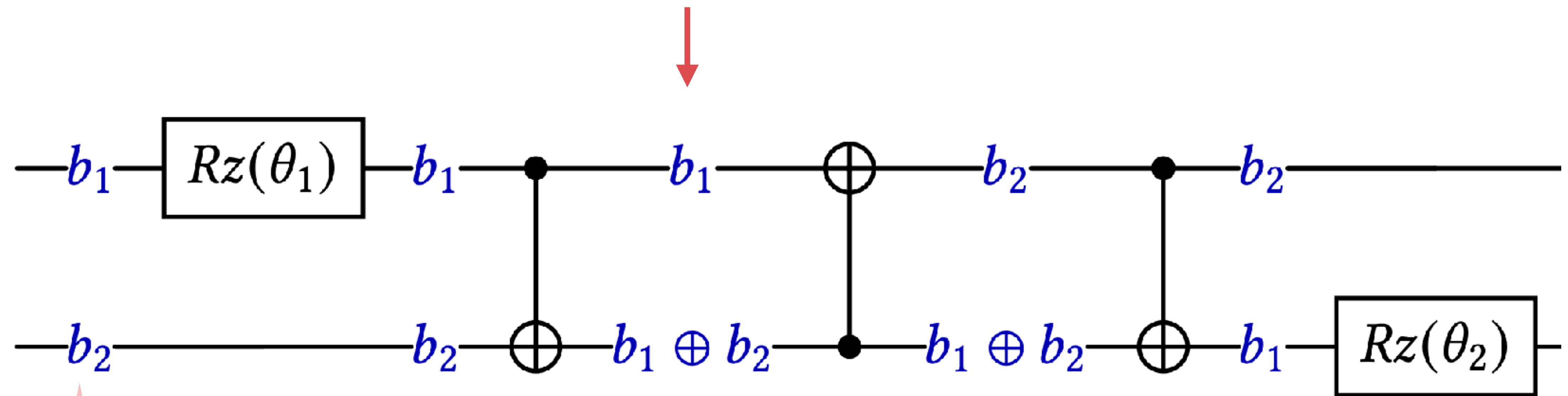


Boolean variables

Symbolic analysis

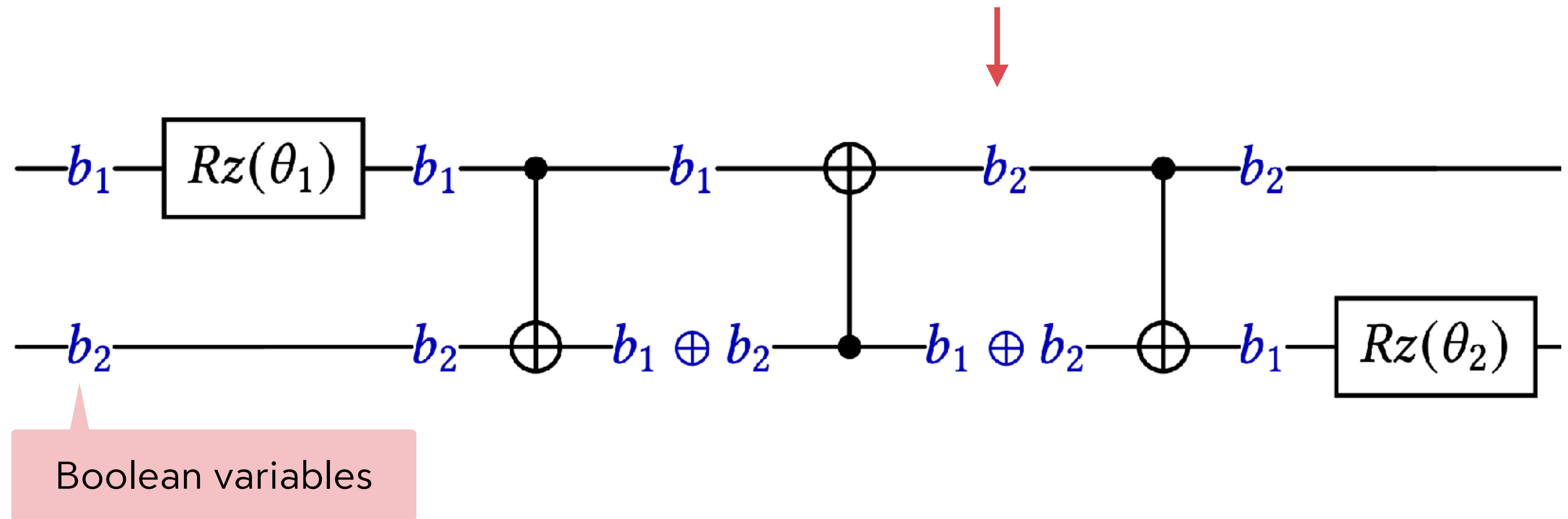


Symbolic analysis

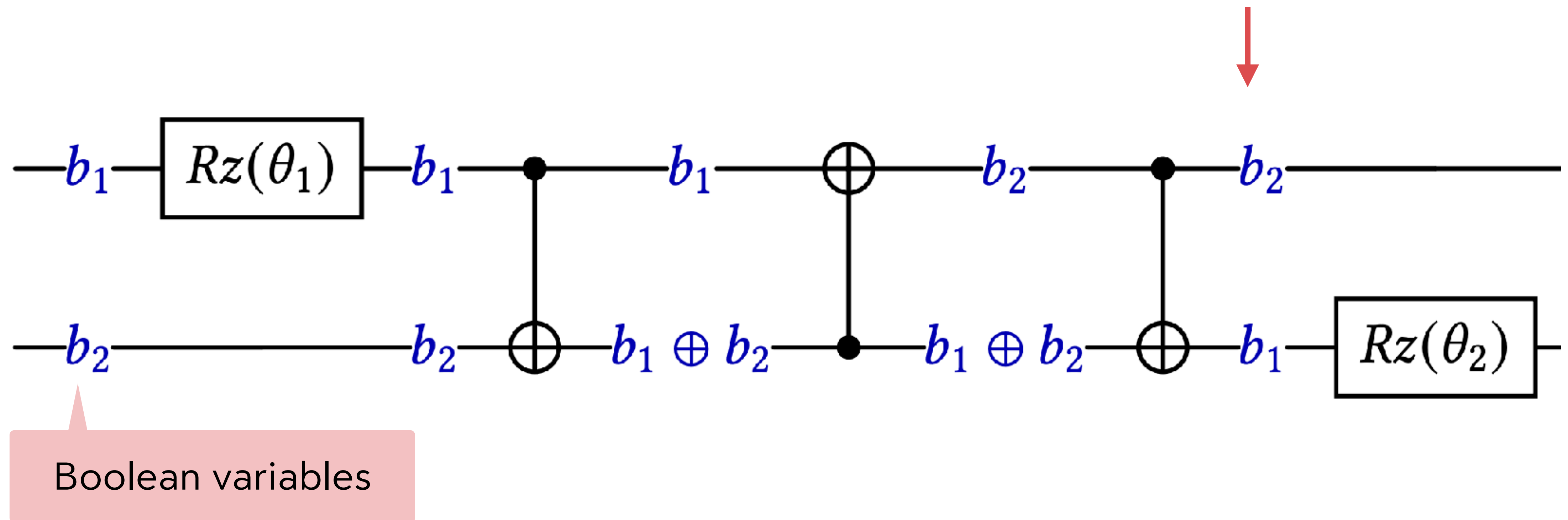


Boolean variables

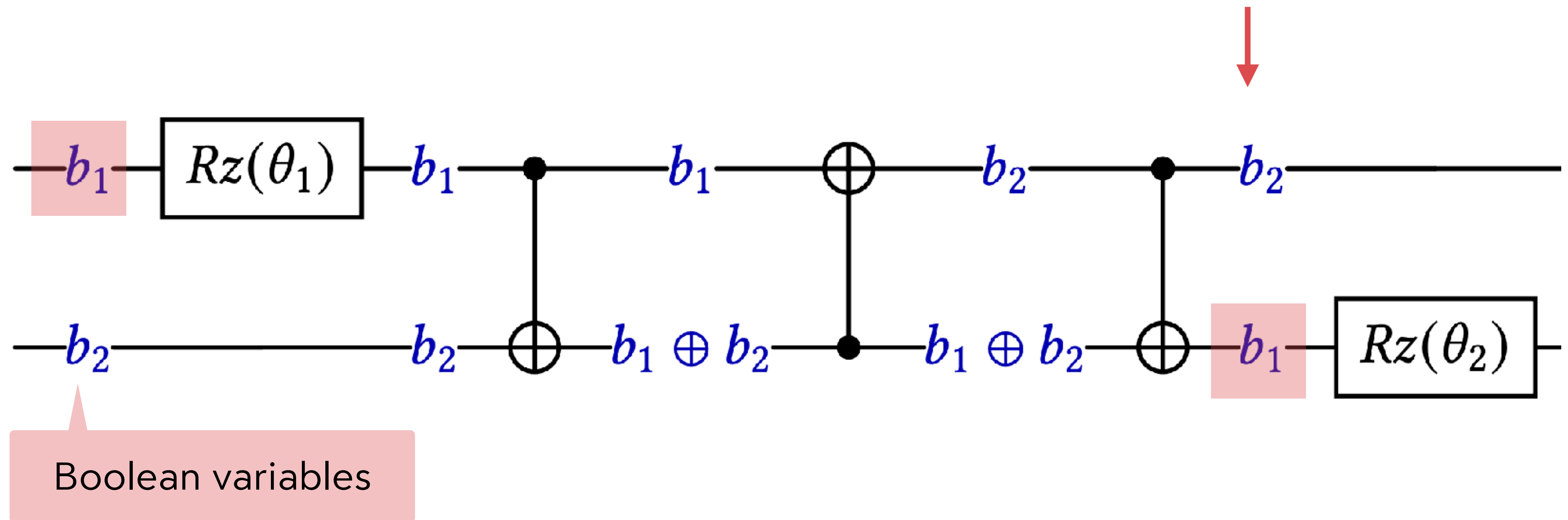
Symbolic analysis



Symbolic analysis



Symbolic analysis



Symbolic analysis is an **overapproximation**

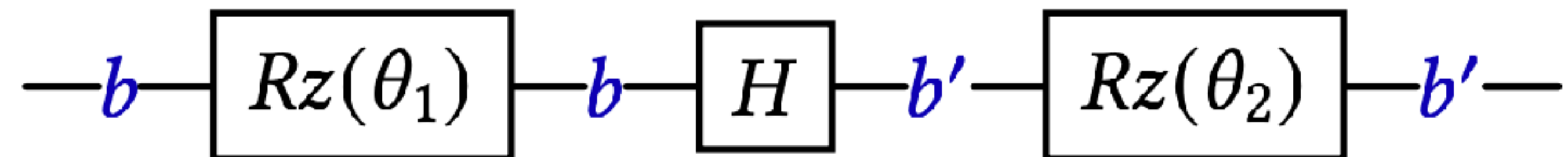
does not track amplitudes

overapproximates support

Symbolic analysis is an **overapproximation**

does not track amplitudes

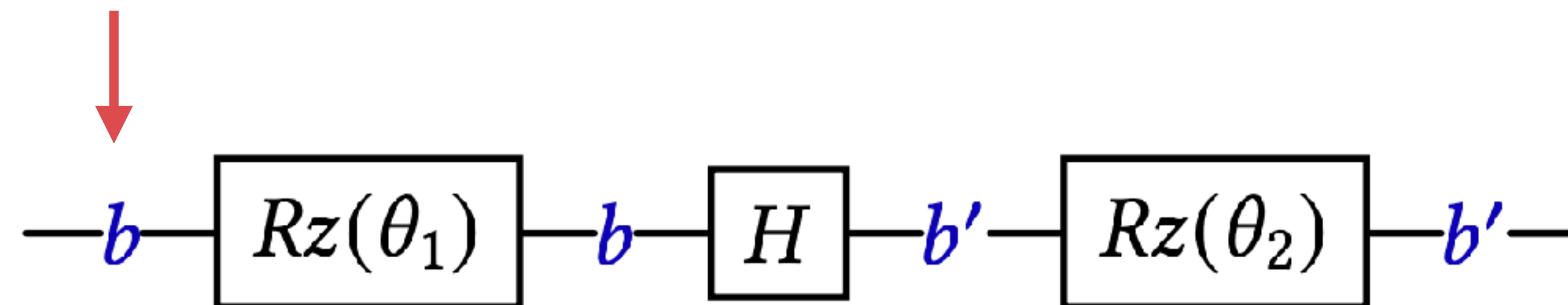
overapproximates support



Symbolic analysis is an **overapproximation**

does not track amplitudes

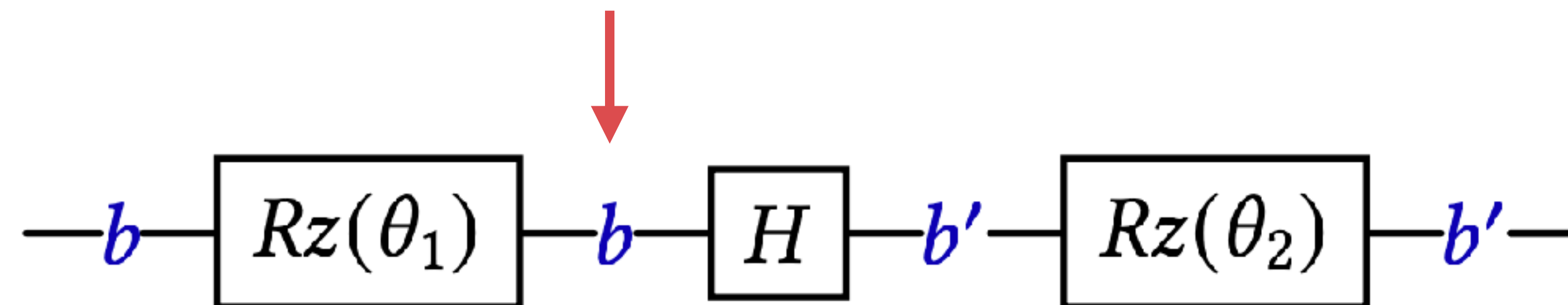
overapproximates support



Symbolic analysis is an **overapproximation**

does not track amplitudes

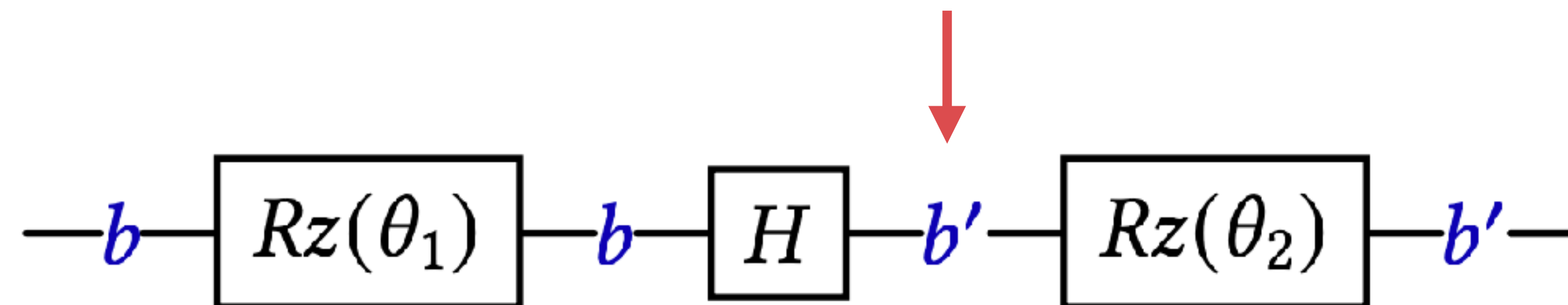
overapproximates support



Symbolic analysis is an **overapproximation**

does not track amplitudes

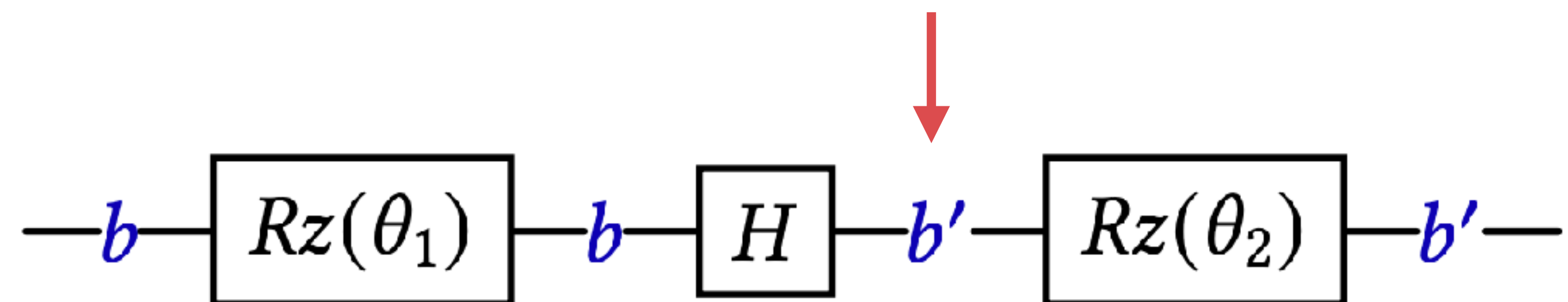
overapproximates support



Symbolic analysis is an **overapproximation**

does not track amplitudes

overapproximates support

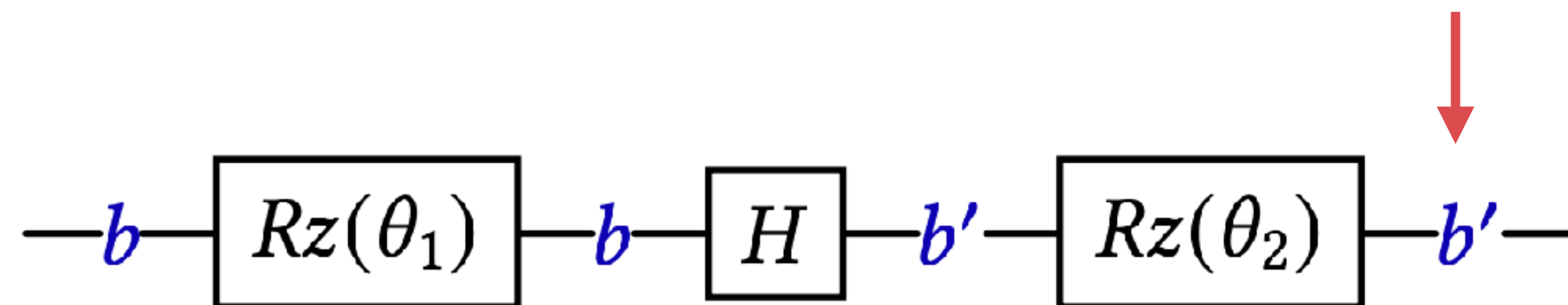


fresh symbolic variable

Symbolic analysis is an **overapproximation**

does not track amplitudes

overapproximates support



fresh symbolic variable

Challenges with symbolic analysis

worst-case: parity formulas can be linear in the size of the circuit

- and we're constantly checking parity equivalence

Challenges with symbolic analysis

worst-case: parity formulas can be linear in the size of the circuit

- and we're constantly checking parity equivalence

complexity: quadratic in circuit size

Idea: Parity equivalence is fast

$$v_0 \oplus v_1$$

$$v_0 \oplus v_2$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

assign **random** a bitstring for each variable

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

assign **random** a bitstring for each variable

$$1011 \oplus 1100$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

assign **random** a bitstring for each variable

$$1011 \oplus 1100$$

$$1011 \oplus 0010$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

assign **random** a bitstring for each variable

$$1011 \oplus 1100$$

$$1011 \oplus 0010$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

assign **random** a bitstring for each variable

$$1011 \oplus 1100 \neq 1011 \oplus 0010$$

Idea: Parity equivalence is fast

$$v_0 \oplus v_1 \neq v_0 \oplus v_2$$

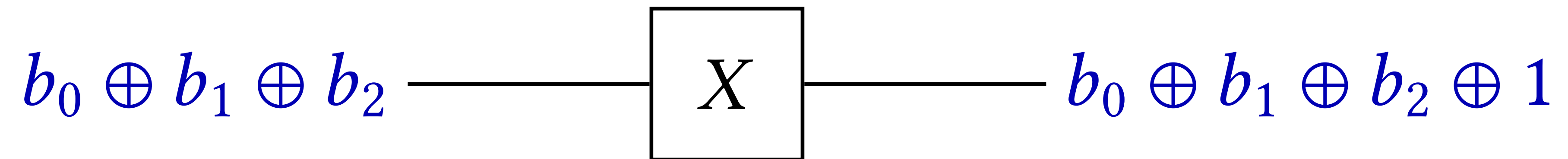
assign **random** a bitstring for each variable

$$1011 \oplus 1100 \neq 1011 \oplus 0010$$

probability of a **false positive** is at most $1/16$
generally, 2^{-k}

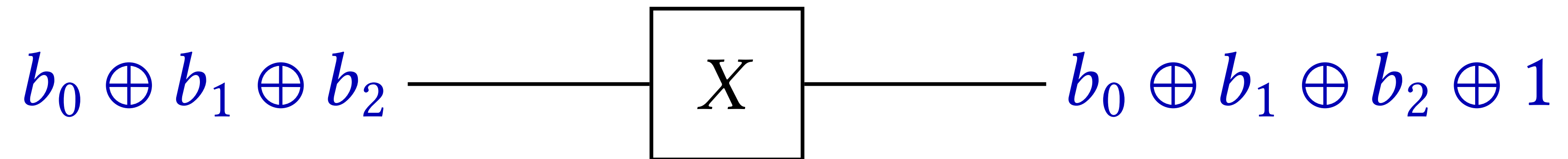
Randomized analysis

symbolic program analysis

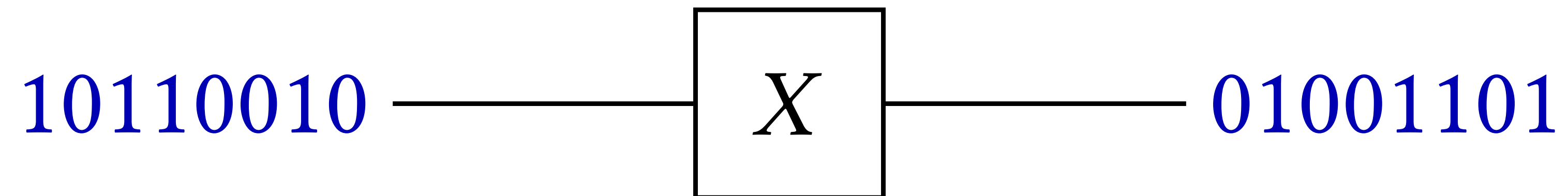


Randomized analysis

symbolic program analysis

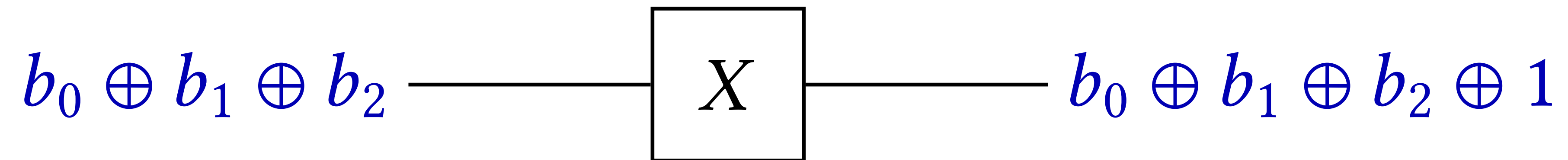


randomized program analysis

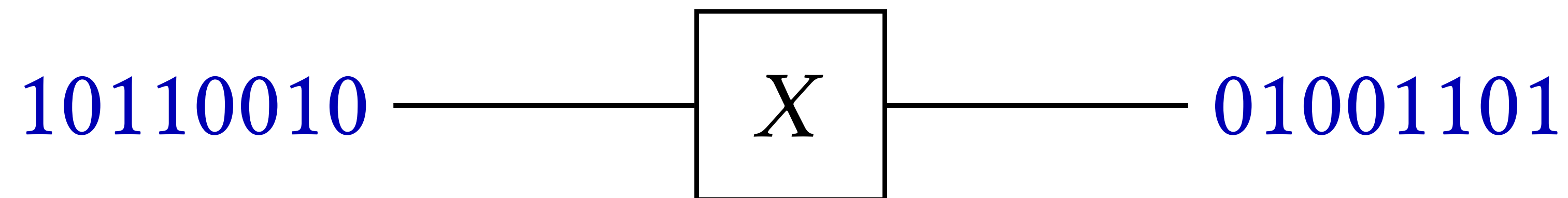


Randomized analysis

symbolic program analysis



randomized program analysis



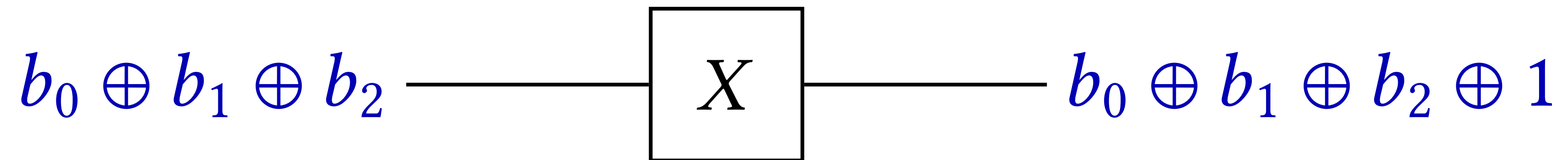
**we are overapproximating the support,
but we have a small probability of error**

by

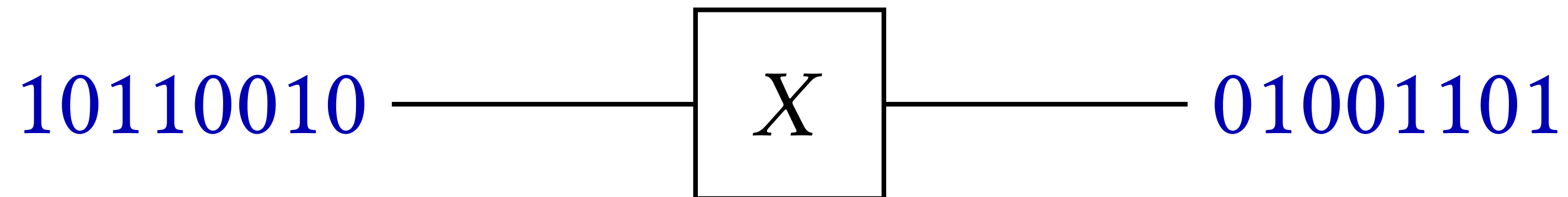
Sumit Gulwani

Randomized analysis

symbolic program analysis

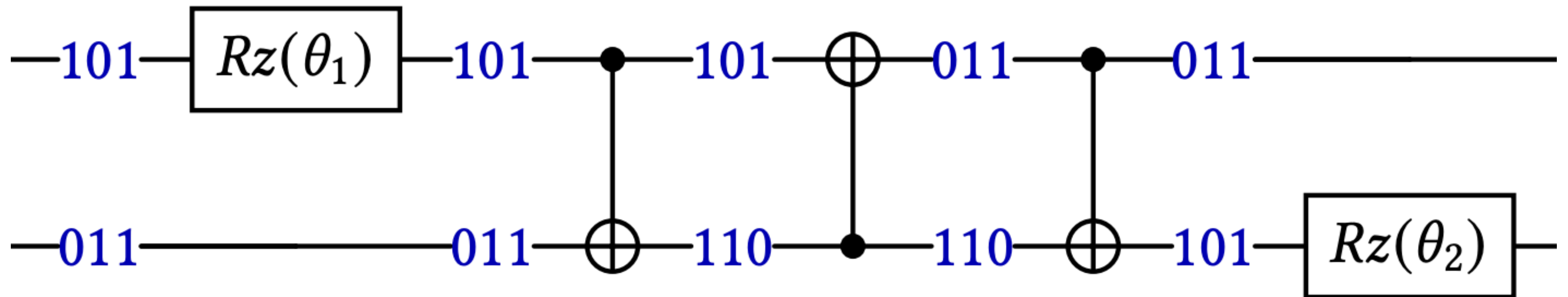


randomized program analysis



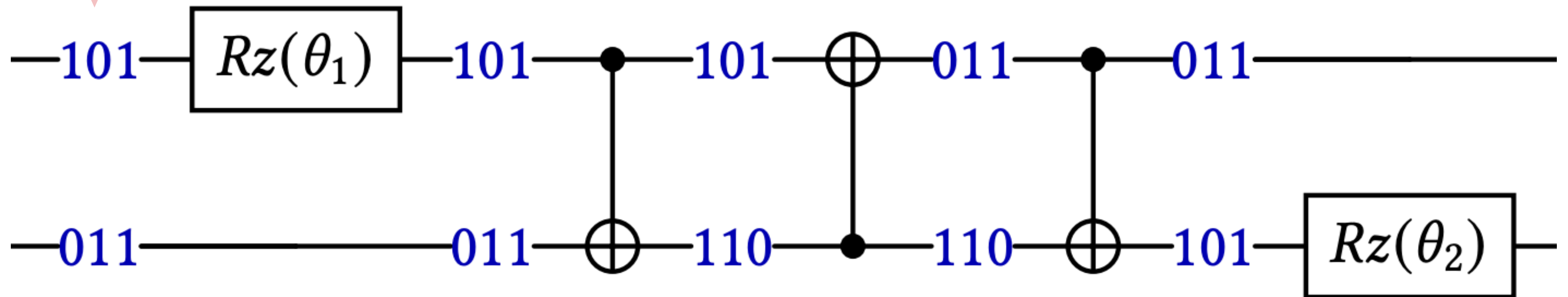
**we are overapproximating the support,
but we have a small probability of error**

Randomized phase folding



Randomized phase folding

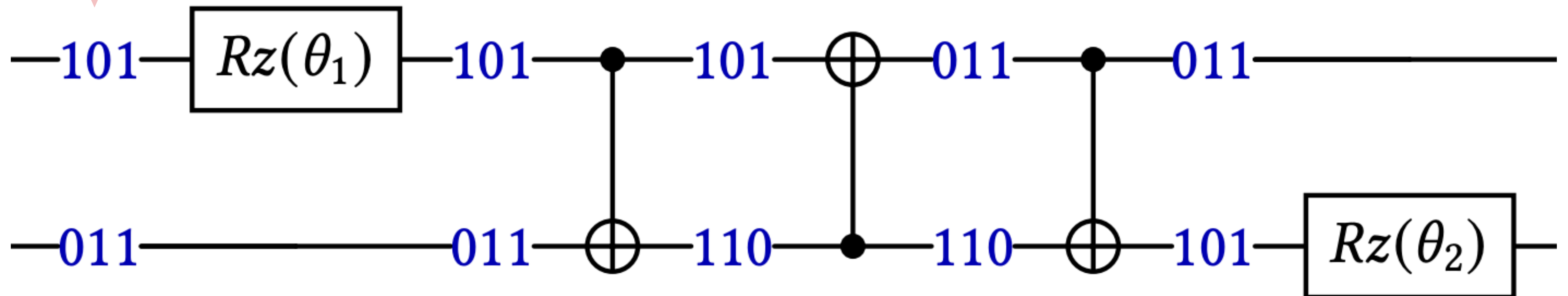
initialize qubits with random
bitstrings



Randomized phase folding

a **linear** algorithm for a fixed bitwidth

initialize qubits with **random** bitstrings



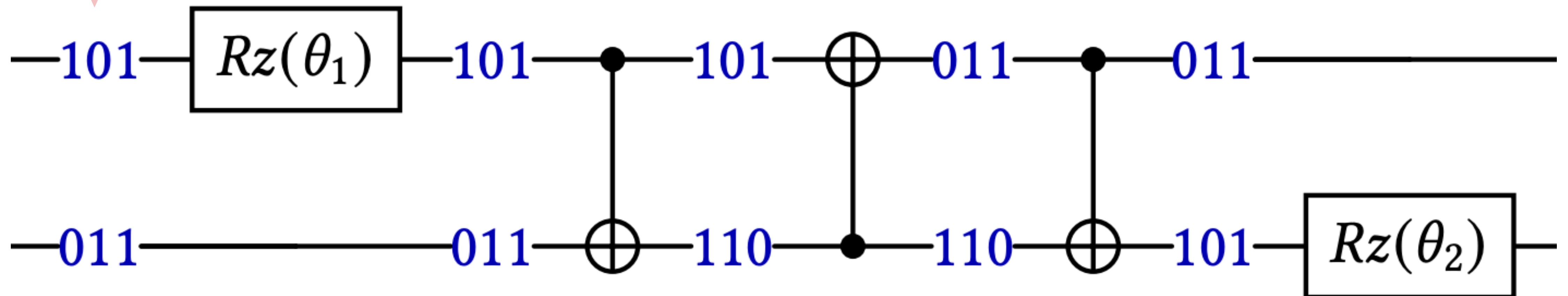
Randomized phase folding

a **linear** algorithm for a fixed bitwidth

128 bits is all you need

failure probability is 10^{-15} for circuits 1 trillion rotations!

initialize qubits with random
bitstrings



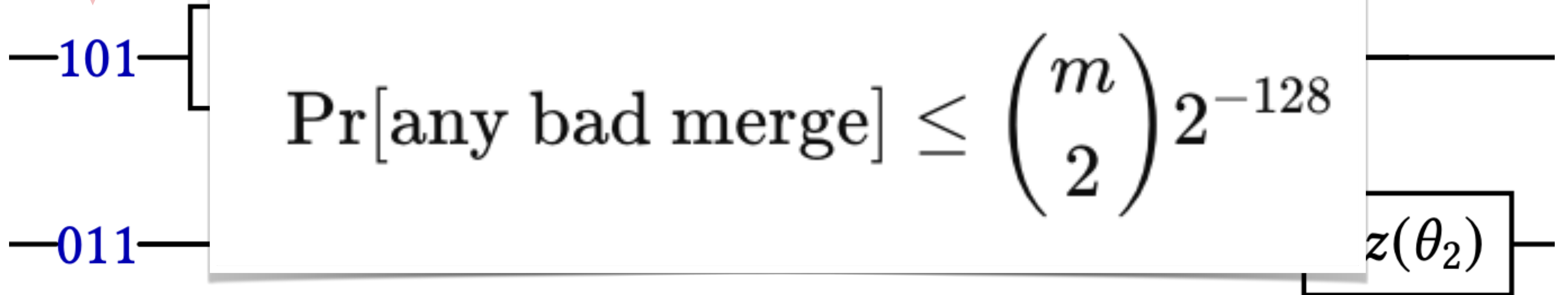
Randomized phase folding

a **linear** algorithm for a fixed bitwidth

128 bits is all you need

failure probability is 10^{-15} for circuits 1 trillion rotations!

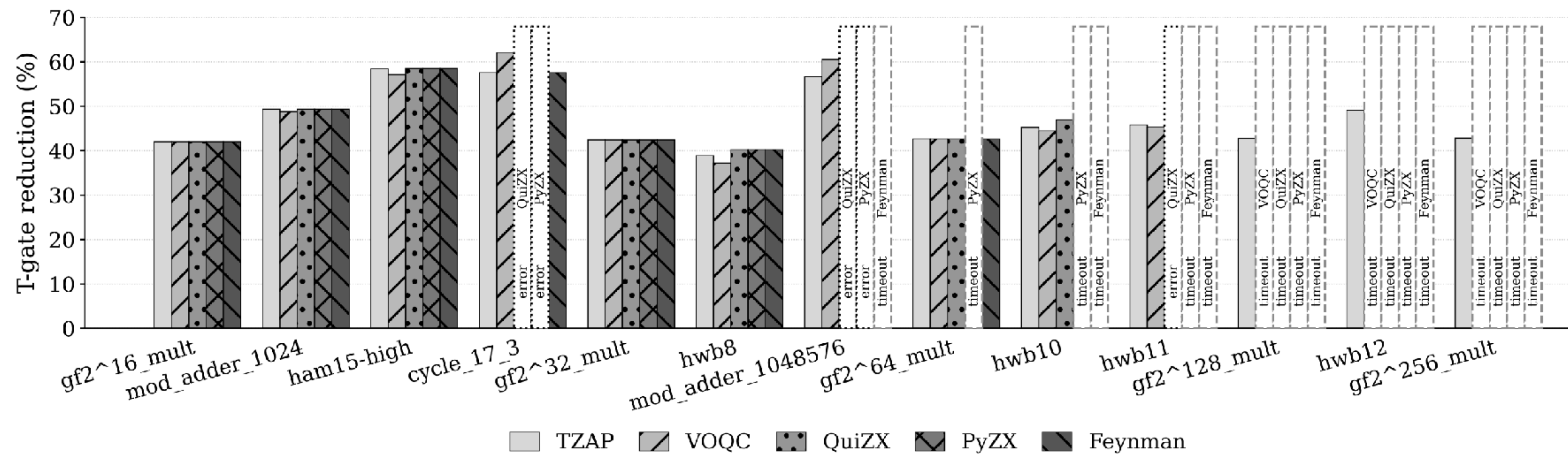
initialize qubits with random
bitstrings



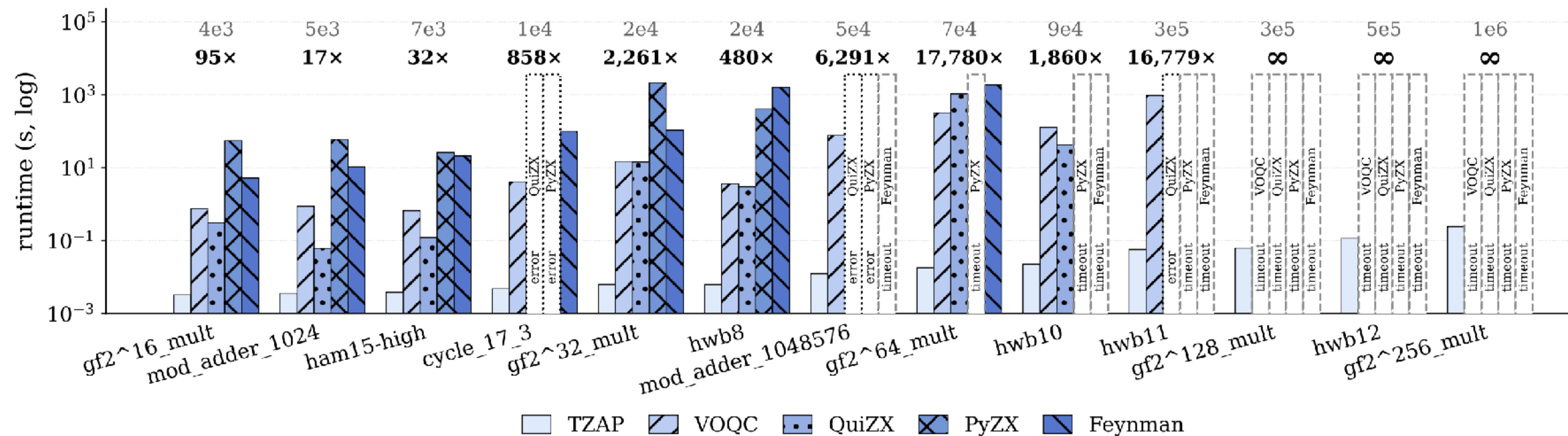
Quantum circuit diagram showing two qubits initialized to 101 and 011. A gate labeled $z(\theta_2)$ is applied to the second qubit. A large white box with a grey border is overlaid on the circuit, containing the inequality:

$$\Pr[\text{any bad merge}] \leq \binom{m}{2} 2^{-128}$$

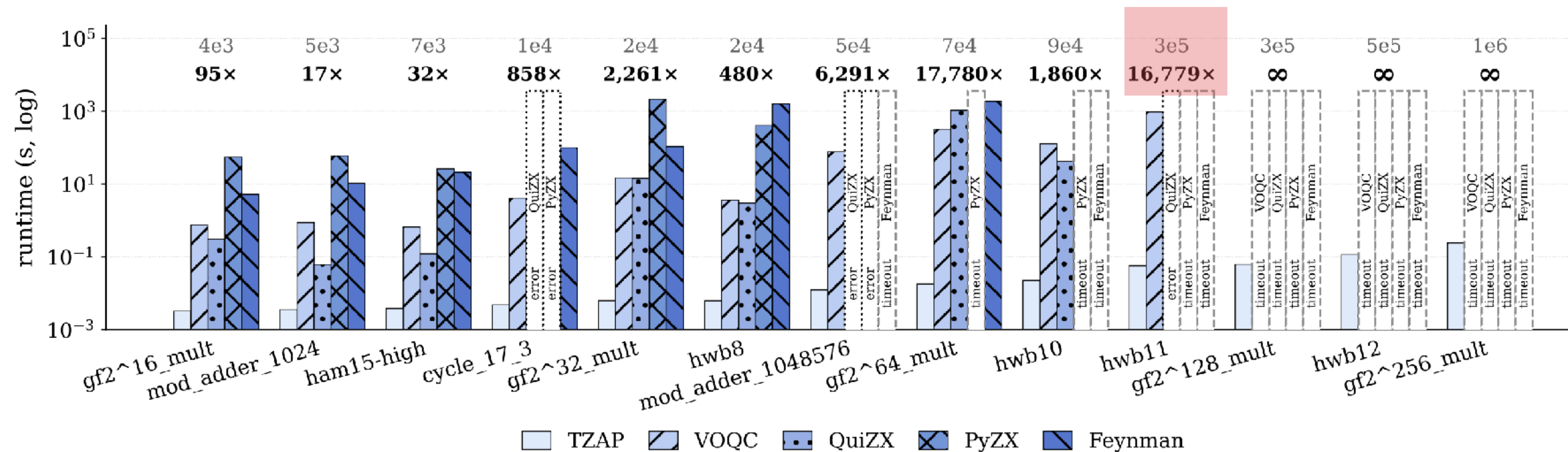
Eval **A**: Strong gate reduction



Eval **B**: Orders of magnitude speedup



Eval **B**: Orders of magnitude speedup



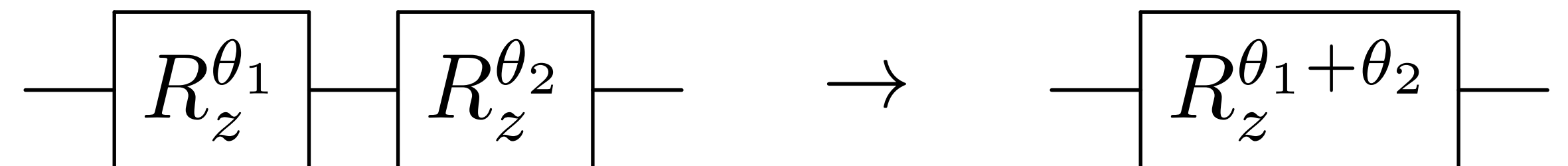
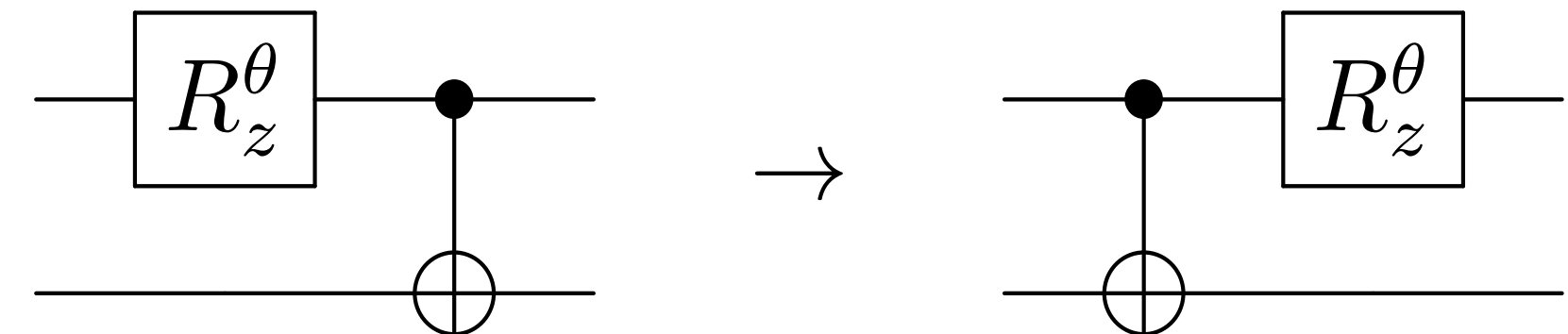
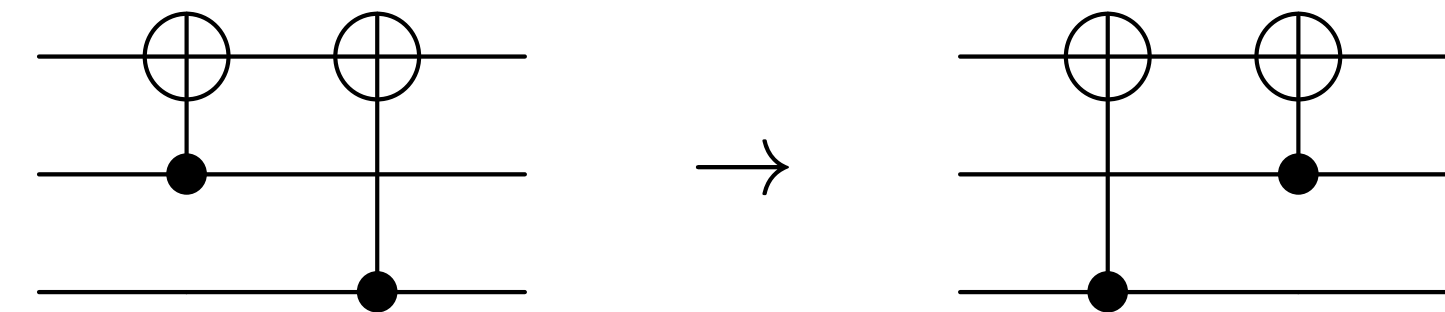
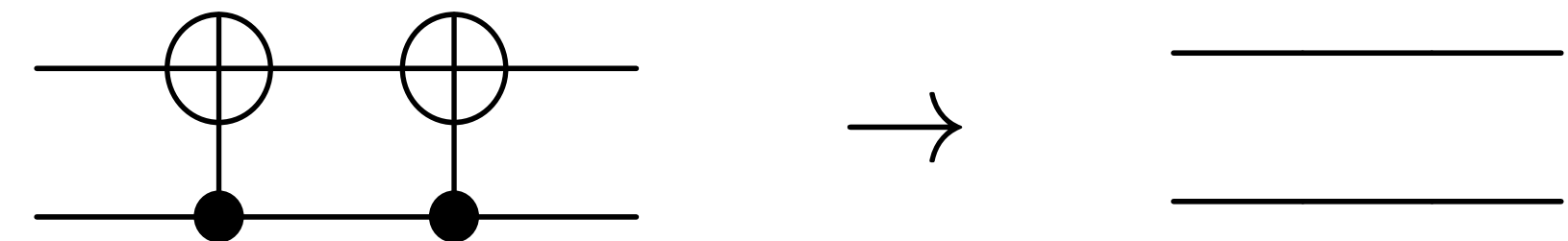
Chapter 2

Fast superoptimization

How do superoptimizers work?

How do superoptimizers work?

start with a set of **rewrite rules**

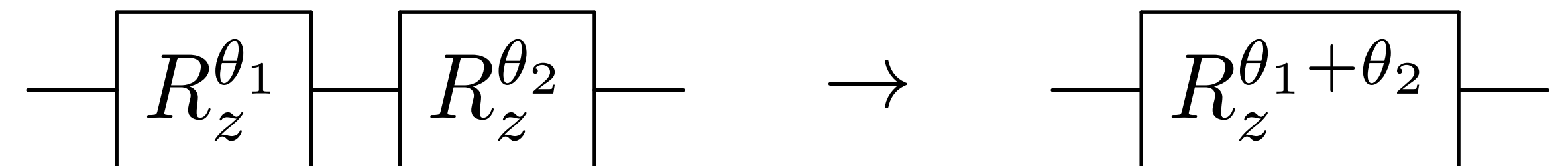
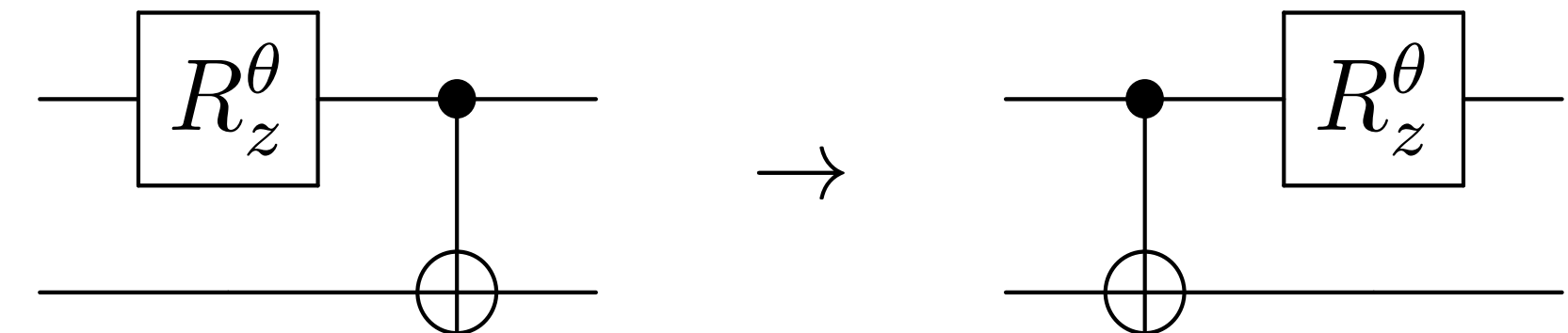
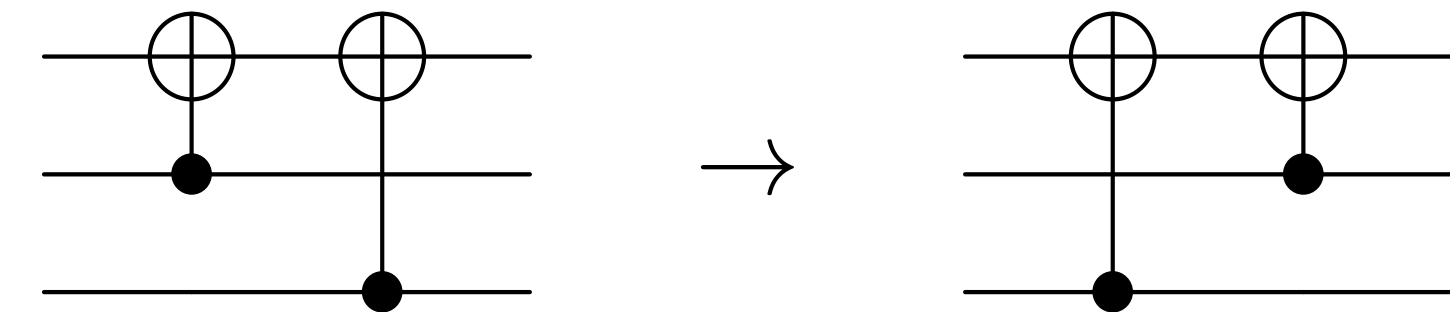
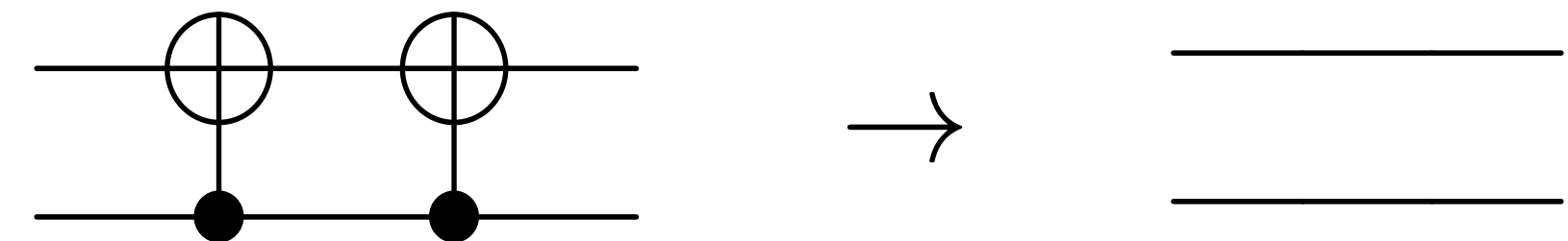


How do superoptimizers work?

start with a set of **rewrite rules**

apply to circuit in **different possible orders**

- use RL, search, equality saturation, ...



How do superoptimizers work?

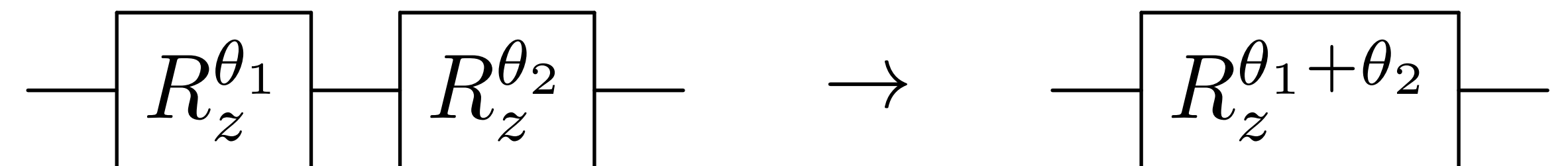
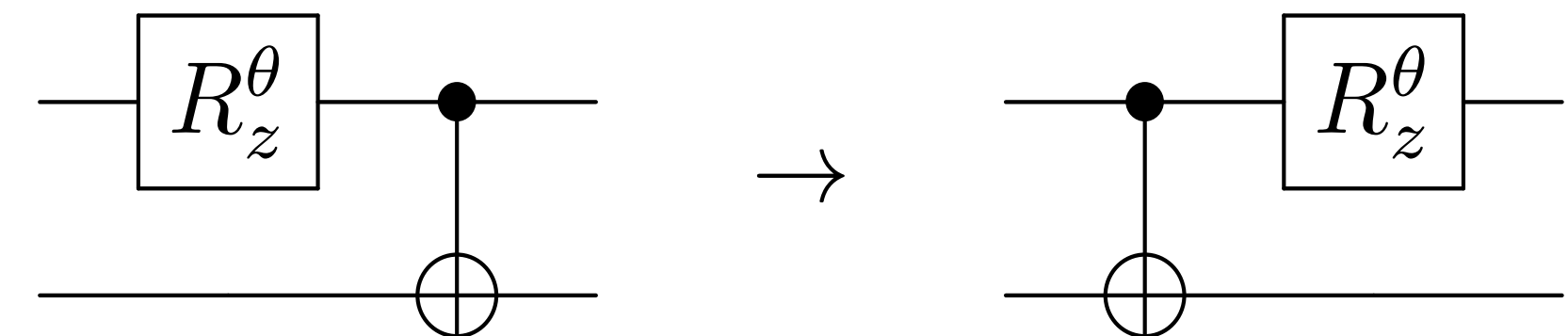
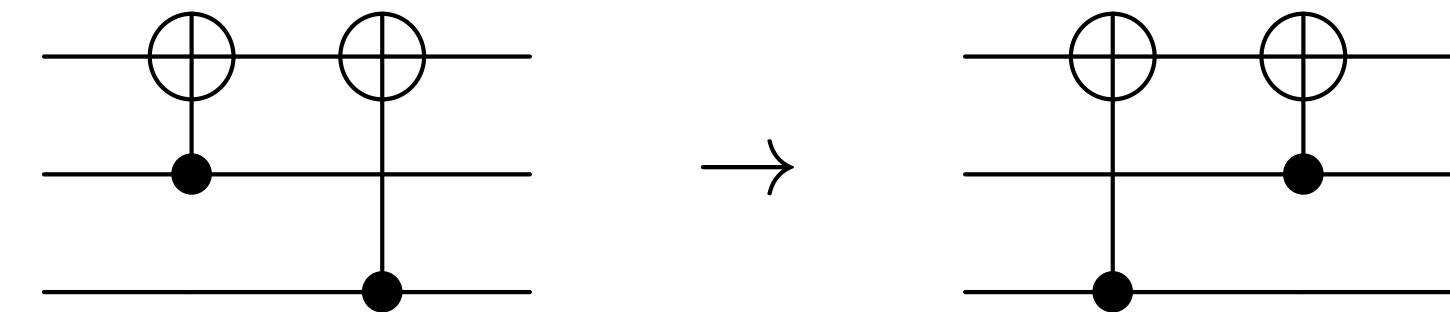
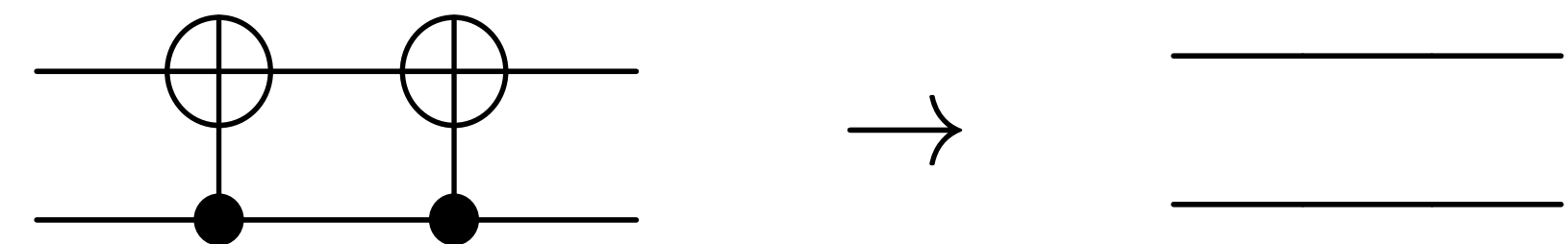
start with a set of **rewrite rules**

apply to circuit in **different possible orders**

- use RL, search, equality saturation, ...

what are the issues?

- search space is hopelessly **enormous**
- rules are **syntactic**



A semantic approach to rewriting

A semantic approach to rewriting

build equivalence classes of circuits with minimal representatives

equivalence classes of circuits

c_1	c_2	$\dots$	
c_3	c_9	c_4	$\dots$

A semantic approach to rewriting

build equivalence classes of circuits with minimal representatives

MURM: minimal unitary representative map

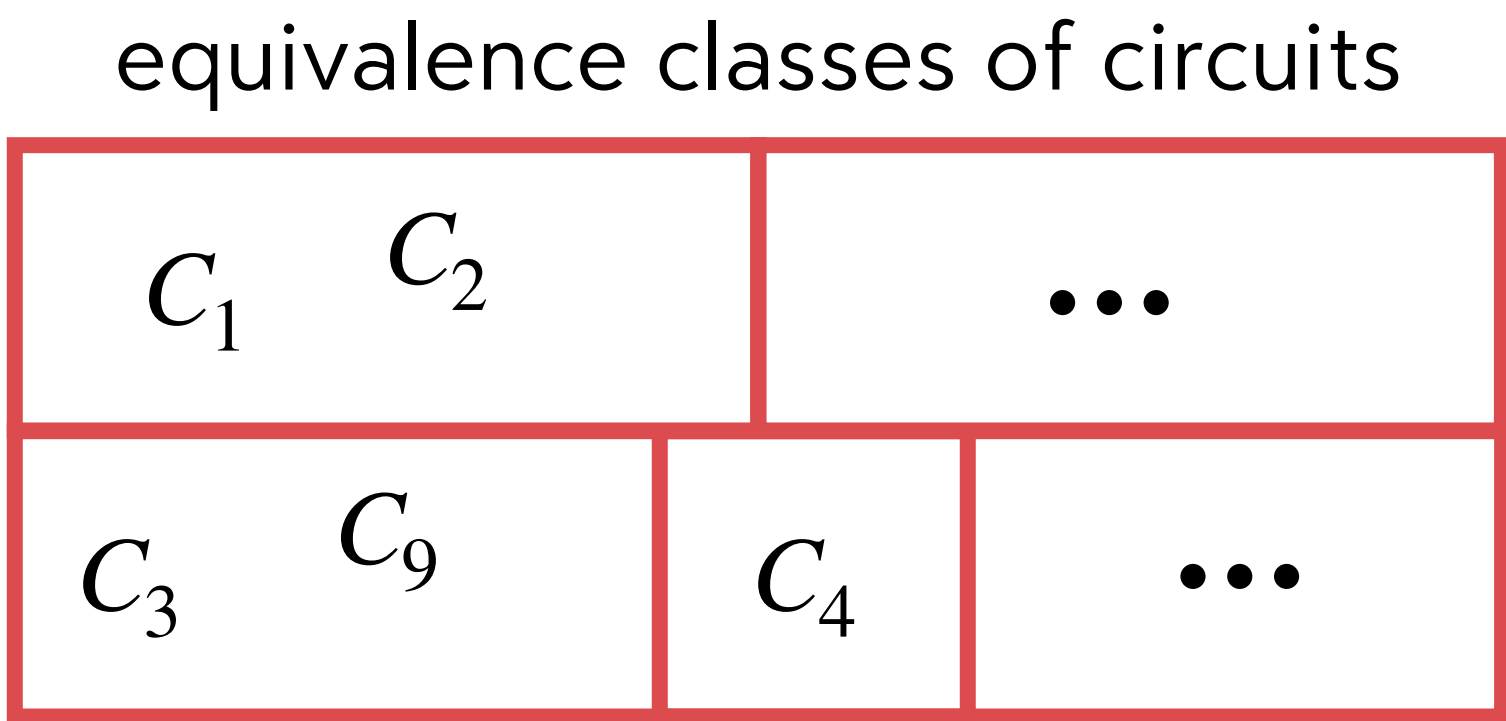
equivalence classes of circuits

C_1	C_2	$\dots$	
C_3	C_9	C_4	$\dots$

A semantic approach to rewriting

build equivalence classes of circuits with minimal representatives

MURM: minimal unitary representative map



MURM

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

basic MURM synthesis algorithm

- enumerate circuits in breadth-first manner

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

basic MURM synthesis algorithm

- enumerate circuits in breadth-first manner
- if circuit's unitary not in MURM, insert it

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

basic MURM synthesis algorithm

- enumerate circuits in breadth-first manner
- if circuit's unitary not in MURM, insert it
- **critical**: only explore from circuits in the MURM!

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

basic MURM synthesis algorithm

- enumerate circuits in breadth-first manner
- if circuit's unitary not in MURM, insert it
- **critical**: only explore from circuits in the MURM!

optimizations

- compactly represent circuits using a prefix graph (constant size per circuit)

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Constructing MURMs

basic MURM synthesis algorithm

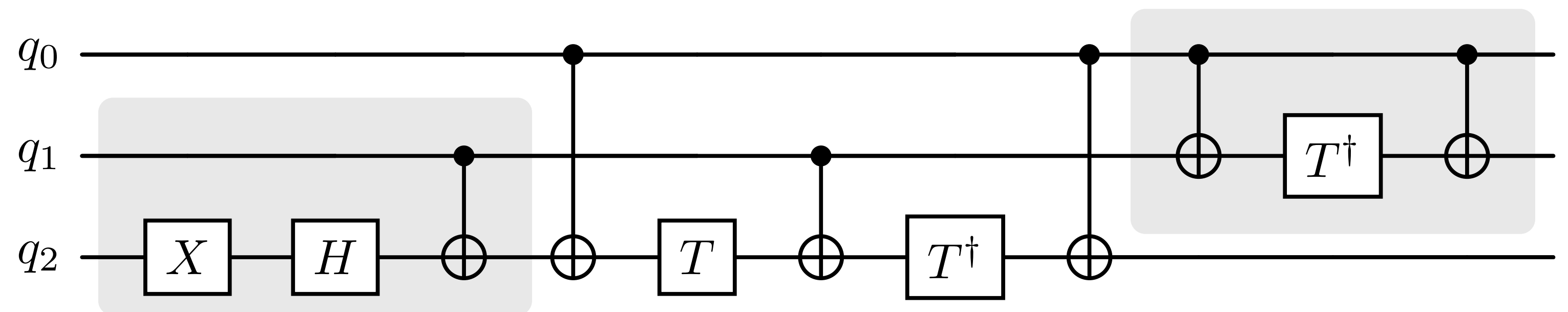
- enumerate circuits in breadth-first manner
- if circuit's unitary not in MURM, insert it
- **critical**: only explore from circuits in the MURM!

optimizations

- compactly represent circuits using a prefix graph (constant size per circuit)
- don't store unitaries, store hashes

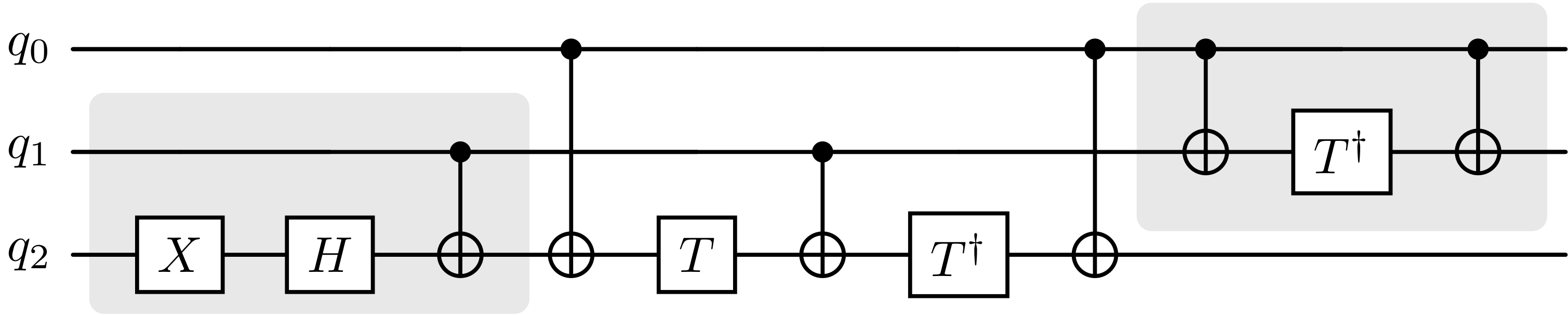
<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

Optimization with MURMs



Optimization with MURMs

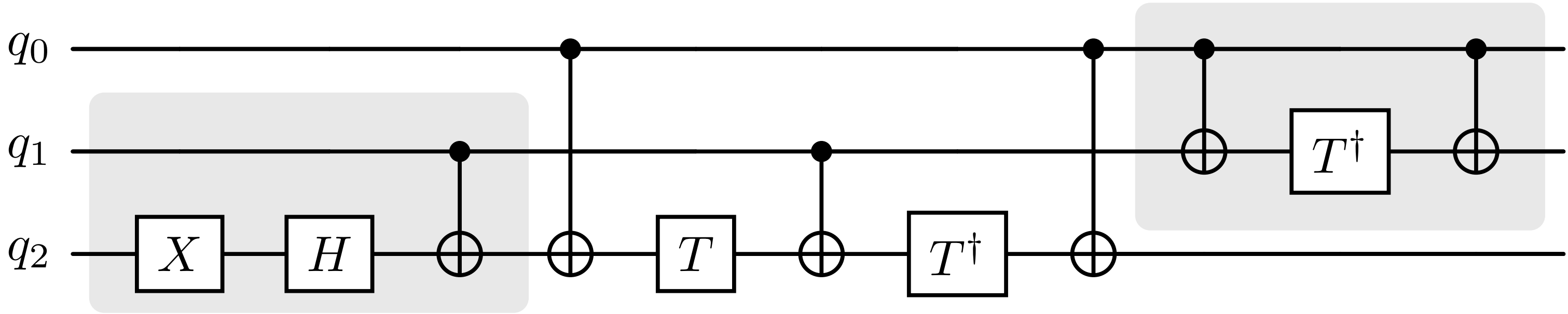
<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
...	...



Optimization with MURMs

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
$\dots$	$\dots$

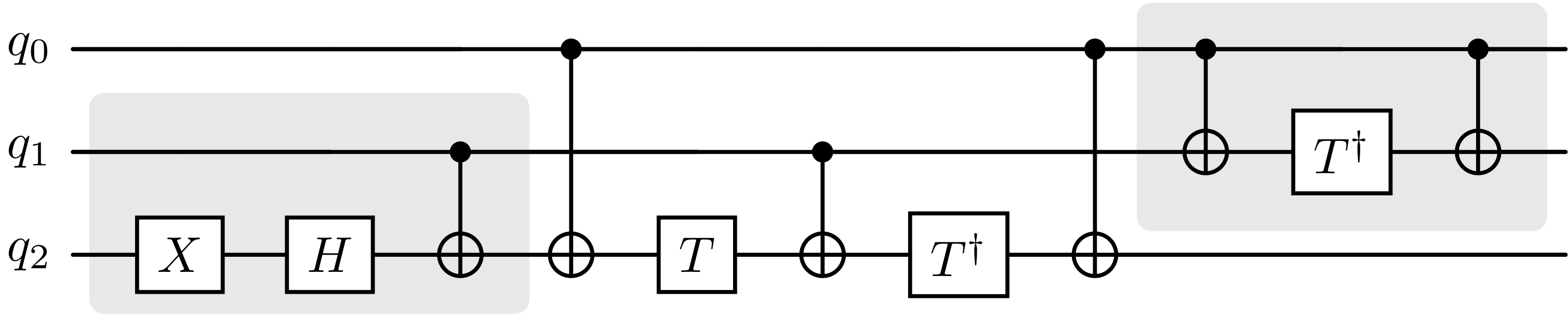
linear scan through circuit
construct bounded subcircuits, and query MURM



Optimization with MURMs

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
...	...

linear scan through circuit
construct bounded subcircuits, and query MURM

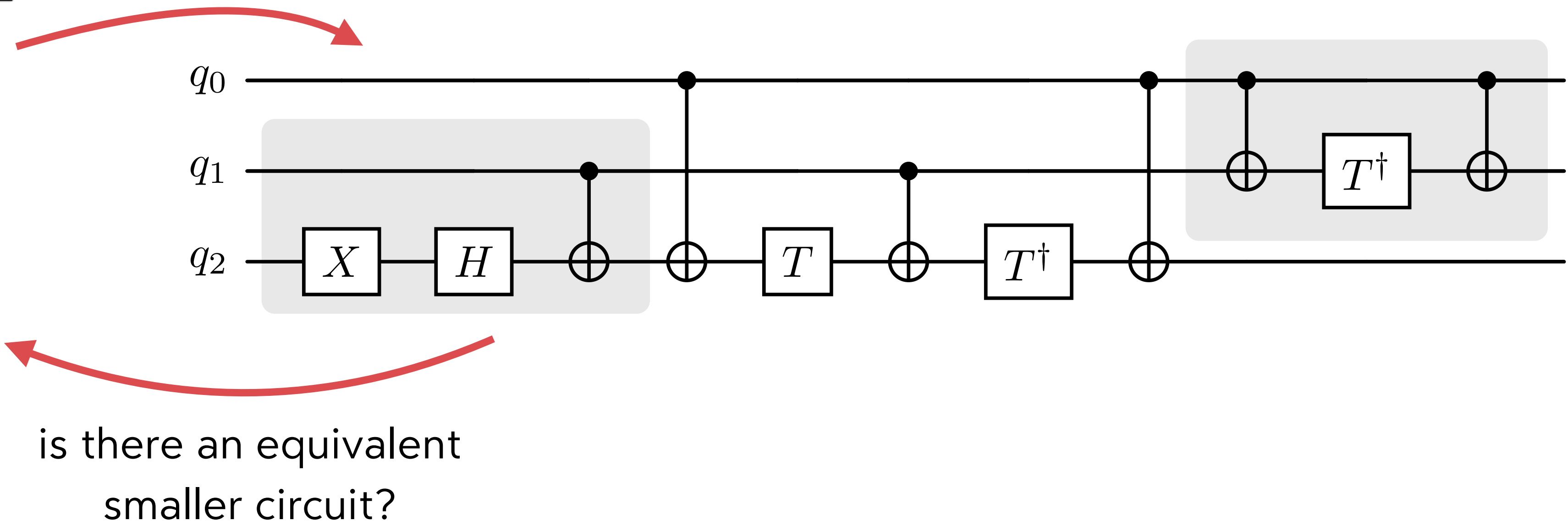


is there an equivalent
smaller circuit?

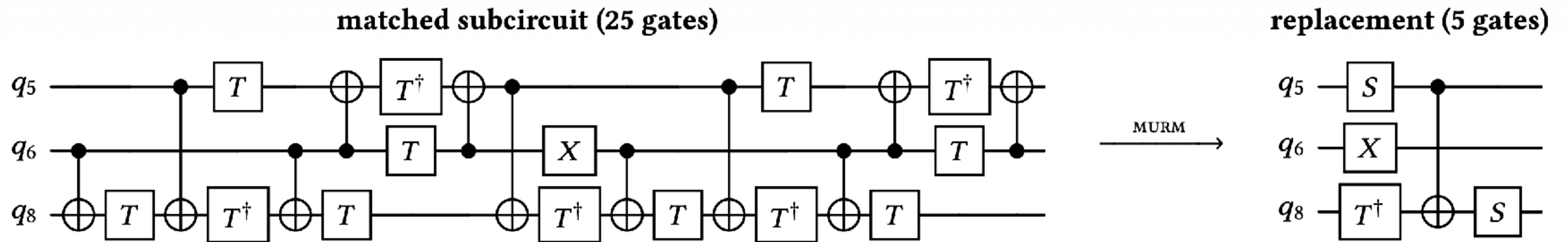
Optimization with MURMs

<i>Projective unitary</i>	<i>Minimal circuit</i>
U_1	C_1
U_2	C_2
U_3	C_3
...	...

linear scan through circuit
construct bounded subcircuits, and query MURM



Ex: Large rewrite



Implementation considerations

Implementation considerations

numerical accuracy

- we deal with Clifford+T here, so all computation is symbolic (cyclotomic ring)

fixpoints!

- the algorithm is very fast
- so we can run it many times, until it can't find new reductions

Two decades ago...

Automatic Generation of Peephole Superoptimizers

Sorav Bansal and Alex Aiken

Computer Systems Lab
Stanford University

{sbansal, aiken}@cs.stanford.edu

Data Structures and Algorithms for Simplifying Reversible Circuits

ADITYA K. PRASAD

Amazon.com Inc.

VIVEK V. SHENDE, IGOR L. MARKOV, and JOHN P. HAYES

University of Michigan

and

KETAN N. PATEL

Qualcomm Inc.

Experimental setup

Experimental setup

diverse set of benchmarks (135 circuits), 1000 to 11 million gates

Experimental setup

diverse set of benchmarks (135 circuits), 1000 to 11 million gates

all kinds of optimizers — traditional and superoptimizers

- 3hrs per tool, 32GB RAM

Experimental setup

diverse set of benchmarks (135 circuits), 1000 to 11 million gates

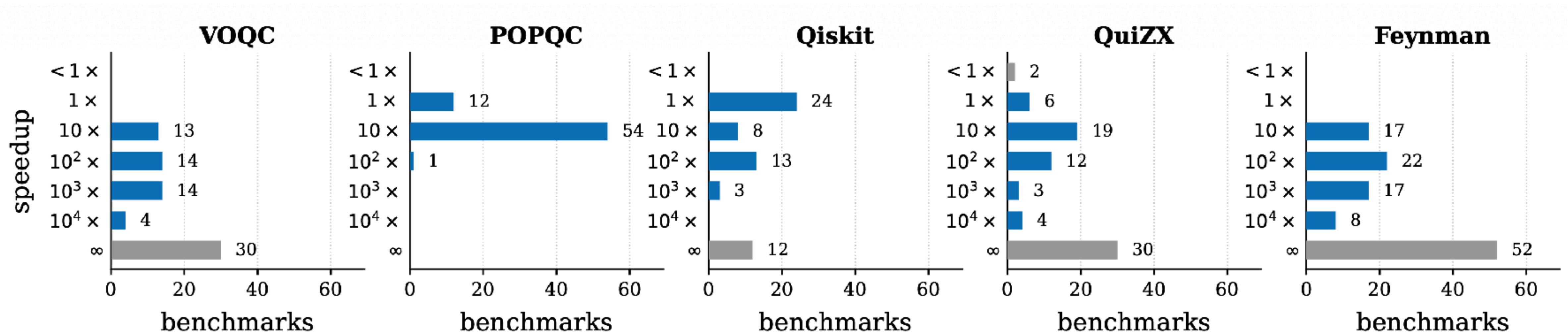
all kinds of optimizers — traditional and superoptimizers

- 3hrs per tool, 32GB RAM

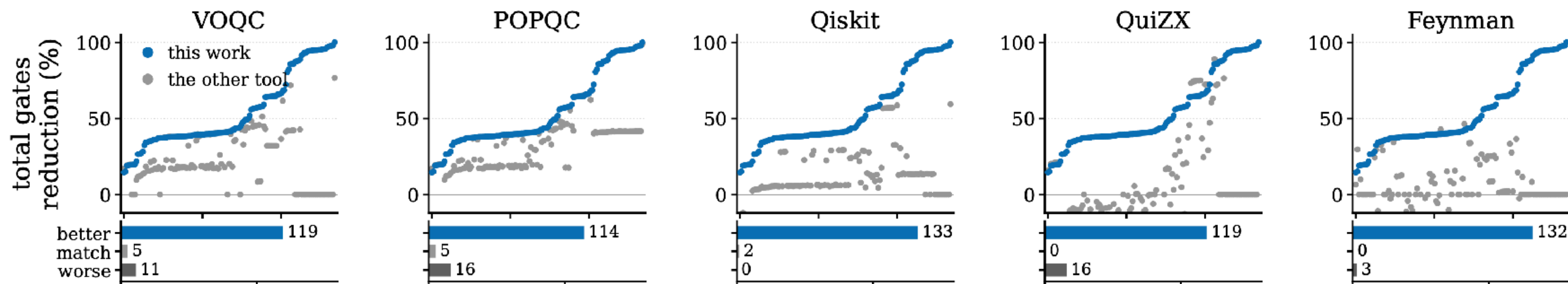
small MURM

- unitaries with up to 3 qubits
- 600,000 entries
- 0.15 seconds to synthesize!

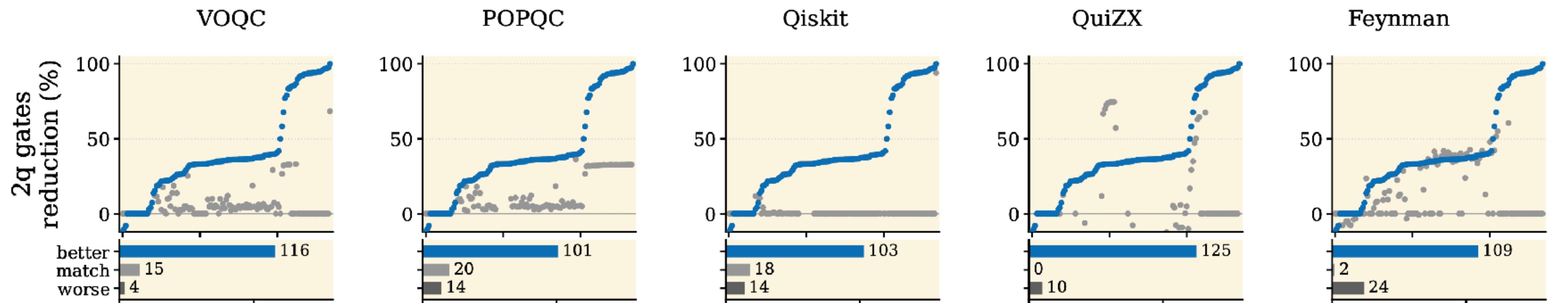
Eval **A**: Order of magnitude speedup



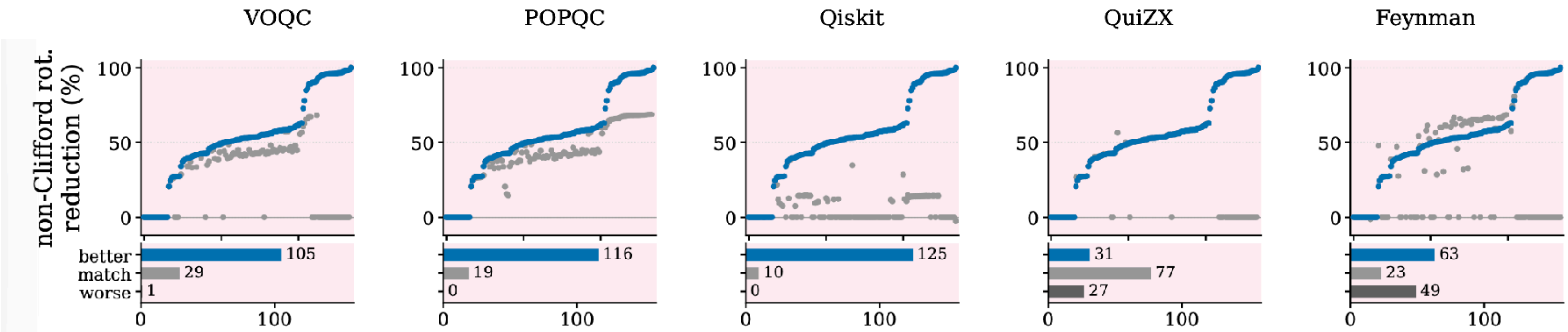
Eval **B**: Significant gate reduction



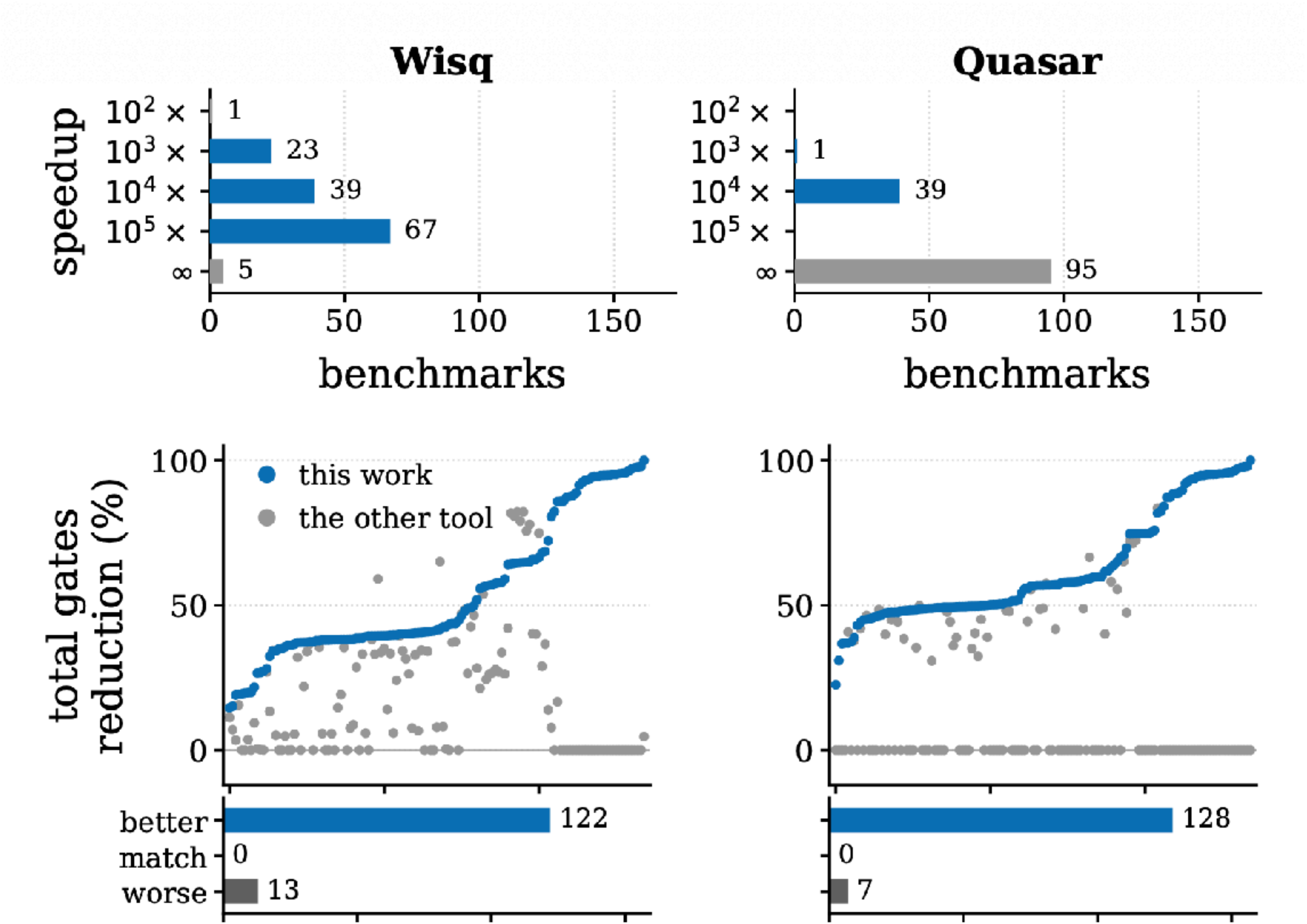
Eval C: Significant 2qubit-gate reduction



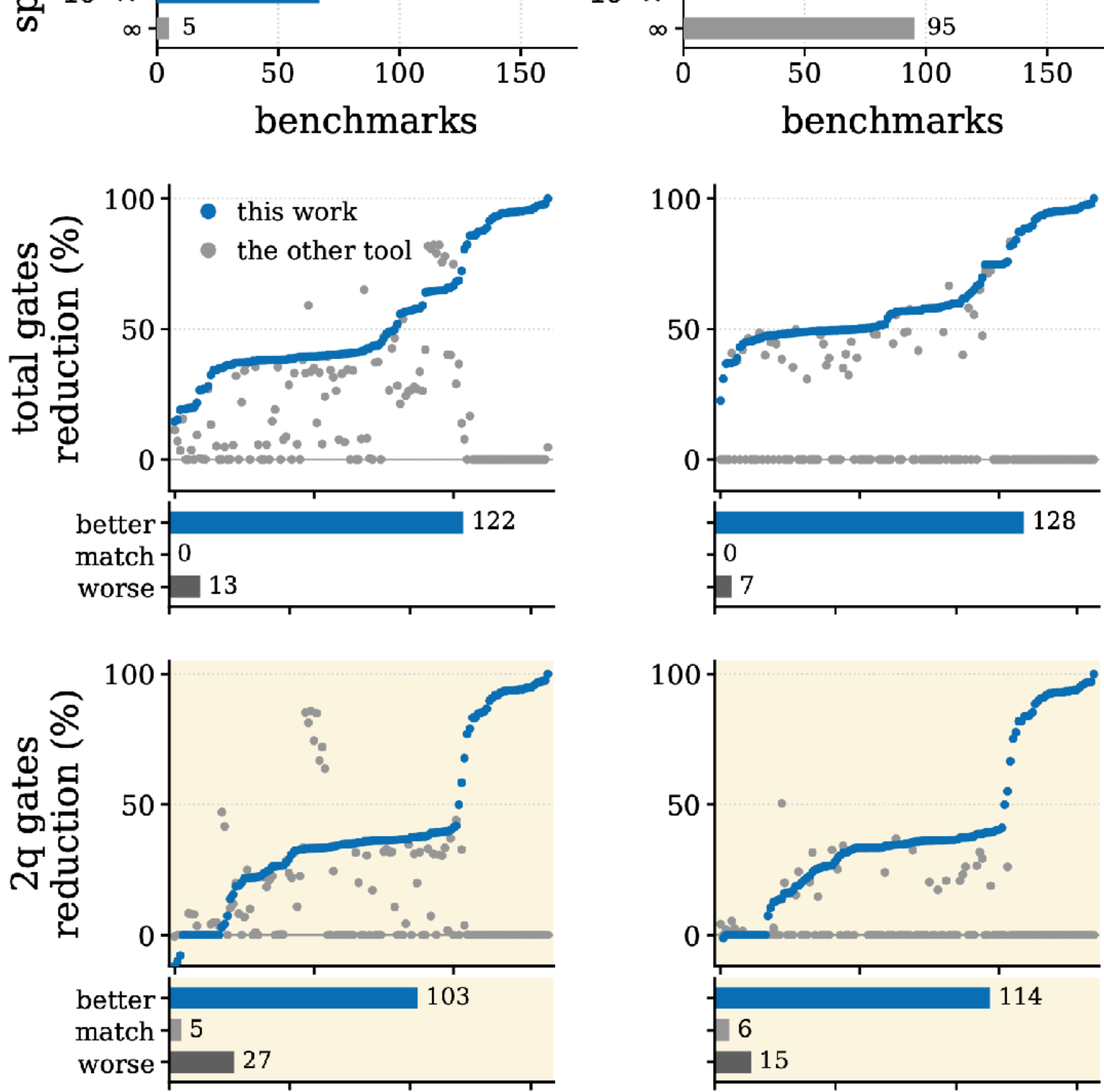
Eval **D**: Significant T-gate reduction



Eval **E**: Outperforms superoptimizers!



Eval E: Ou



Epilogue

On correctness

Formally verified tzap

Formally verified tzap

tzap is written in Rust

Formally verified tzap

tzap is written in Rust

there is also a **Lean** version with formal proofs of correctness

- $\forall C$, $\text{tzap}(C)$ is equivalent to C

```
structure Pass where
  name : String
  run :  $\forall \{n\ m\}, \text{Circuit } n\ m \rightarrow \text{Circuit } n\ m$ 
  correct :  $\forall \{n\ m\} (c : \text{Circuit } n\ m),$ 
    (run c).Equivalent c
```

Formally verified tzap

tzap is written in Rust

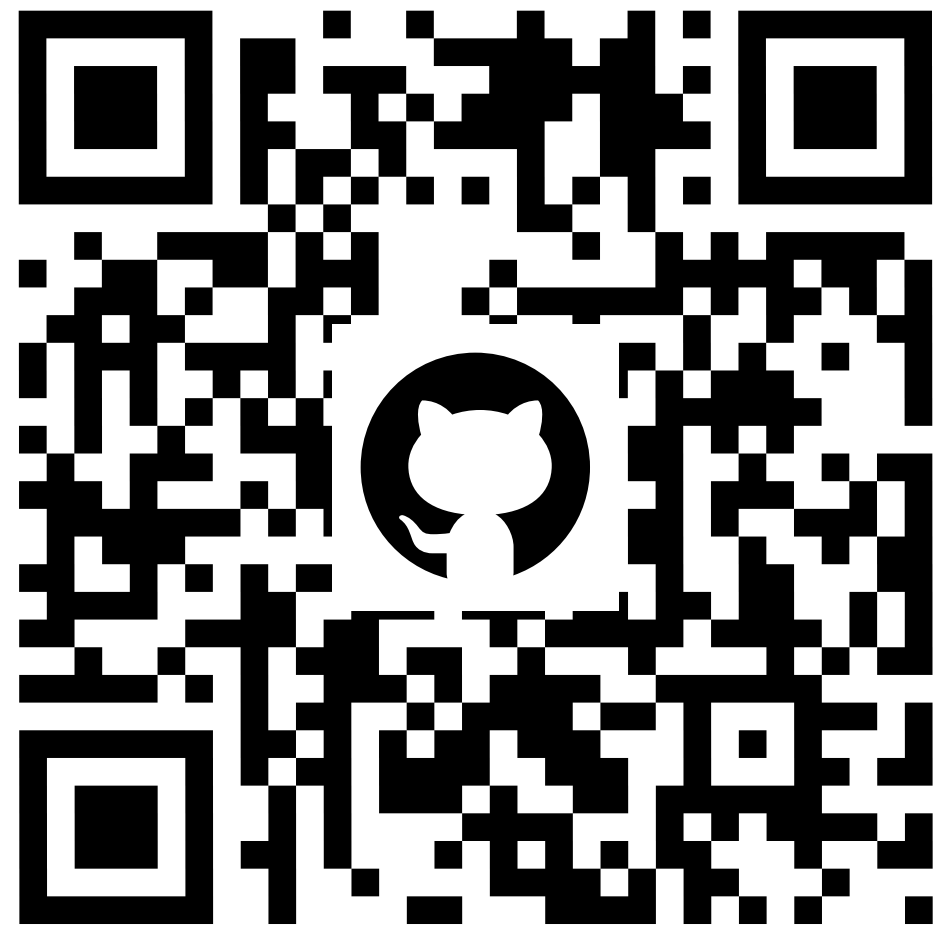
```
structure Pass where
  name : String
  run : ∀ {n m}, Circuit n m → Circuit n m
  correct : ∀ {n m} (c : Circuit n m),
    (run c).Equivalent c
```

there is also a **Lean** version with formal proofs of correctness

- $\forall C$, tzap(C) is equivalent to C

open problems

- can we make the Lean implementation as fast as Rust?
- can we automatically discover new optimization techniques?



github.com/qqq-wisc/tzap

lab projects qqq-wisc.github.io

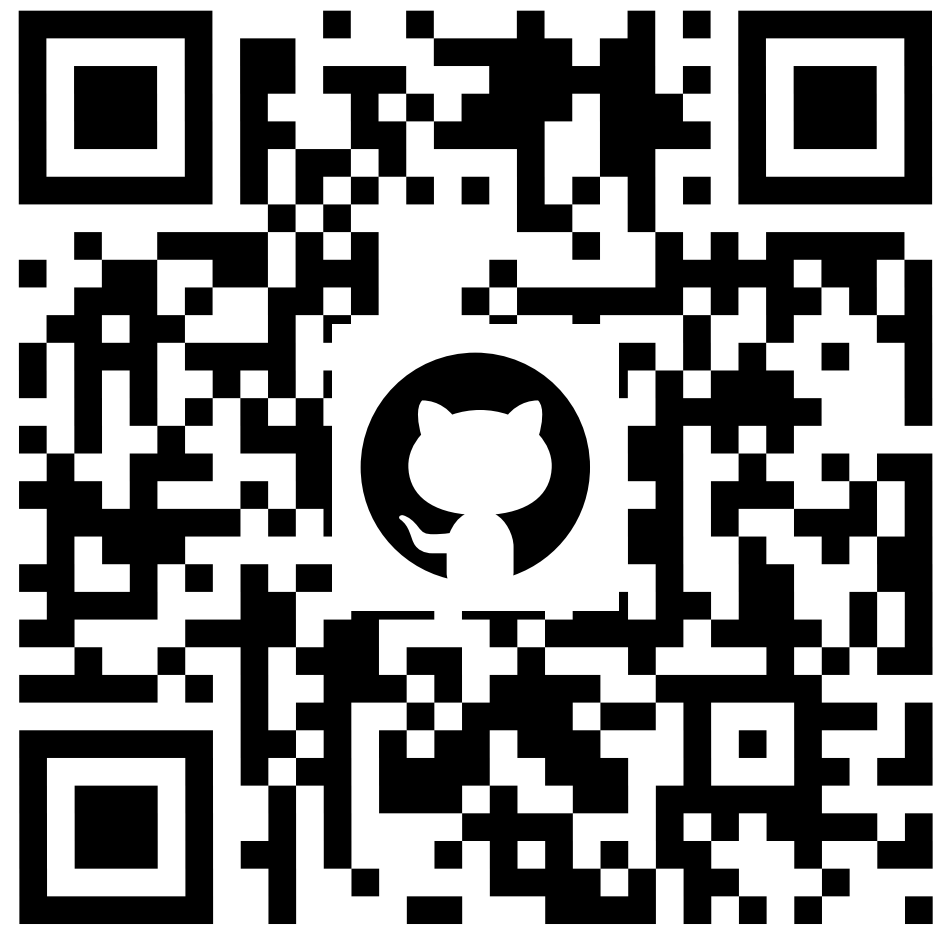


CI **passing** crates.io **v0.5.0** pypi **v0.6.0** Rust Lean 4 license **Apache 2.0** arXiv **2605.13929**

[Installation](#) · [Using tzap](#) | [Qiskit integration](#) · [PennyLane integration](#)

A super fast, Rust-based optimizer for large Clifford+T circuits.

- tzap is state-of-the-art in *speed*, *scalability*, and *gate-count reduction*.
- tzap **minimizes T-count** with a new linear-time phase folding algorithm, based on [this paper](#).
- tzap implements a new and fast **superoptimization** pass.
- A **formally verified** Lean port of tzap is available in [lean](#).



github.com/qqq-wisc/tzap

lab projects qqq-wisc.github.io



CI **passing** crates.io v0.5.0 pypi v0.6.0 Rust Lean 4 license Apache 2.0 arXiv 2605.13929

[Installation](#) · [Using tzap](#) | [Qiskit integration](#) · [PennyLane integration](#)

A super fast, Rust-based optimizer for large Clifford+T circuits.

- tzap is state-of-the-art in *speed*, *scalability*, and *gate-count reduction*.
- tzap **minimizes T-count** with a new linear-time phase folding algorithm, based on [this paper](#).
- tzap implements a new and fast **superoptimization** pass.
- A **formally verified** Lean port of tzap is available in [lean](#).

next steps:

optimizing Pauli-based computation (PBC)

adding classical control

what else?